УНИВЕРЗИТЕТ У НИШУ

ЕЛЕКТРОНСКИ ФАКУЛТЕТ У НИШУ

Веб решење за аутоматско генерисање и управљање правним документима уз асистенцију вештачке интелигенције

Завршни рад

Студијски програм: Електротехника и рачунарство

Модул: Рачунарство и информатика

Ментор:
Доц. др Александар Миленковић

Студент:
Анђела Стојановић
број индекса: 18406

Ниш, новембар 2025. године

УНИВЕРЗИТЕТ У НИШУ
ЕЛЕКТРОНСКИ ФАКУЛТЕТ

Веб решење за аутоматско генерисање и управљање правним документима уз асистенцију вештачке интелигенције

Web-Based Solution for Automatic Generation and Management of Legal Documents with the Assistance of Artificial Intelligence

Завршни рад

Студијски програм: Електротехника и рачунарство
Модул: Рачунарство и информатика

Студент: Анђела Стојановић, број индекса: 18406
Ментор: Доц. др Александар Миленковић

Задатак: *Развој веб решења који омогућава корисницима да генеришу правне документе на основу унетих података, користећи унапред дефинисане шаблоне и асистенцију вештачке интелигенције. Решење функционише по принципу итеративног чаробњака („wizard“) који води корисника кроз процес попуњавања, док се сви генерисани документи чувају у историји корисника ради каснијег приступа и поновне употребе. Коришћене технологије су ASP.NET Core за серверску логику и REST API, ReactJS за израду корисничког интерфејса и Microsoft SQL Server база података. Вештачка интелигенција интегрише се путем OpenAI API сервиса.*

Датум пријаве рада: _____

Датум предаје рада: _____

Датум одбране рада: _____

Комисија за оцену и одбрану:

_____

_____

_____

Ниш, новембар 2025. године

Садржај

1. Увод 2
2. Архитектура и технологије веб решења 3
2.1. Клијентски и серверски део апликације 4
3. ASP.NET 7
3.1. ASP.NET Core 7
3.1.1 ASP.NET Core Web API 8
3.1.2. Сигурност и аутентификација у ASP.NET Core-у 8
3.2. ASP.NET Web API 9
3.2.1. REST Web 10
4. React 12
4.1. Компоненте 12
4.2. JSX и декларативни приказ 12
4.3. Hooks 12
4.4. Управљање стањем 13
4.5. Рутирање 13
4.6. Рад са сервером 13
4.7. React екосистем 13
5. SQL Server 14
5.1. Кључне карактеристике SQL Server-а 14
5.2. SQL и T-SQL 15
6. OpenXML 16
7. OpenAI 17
7.1. Интеграција OpenAI API-ја у .NET апликације 17
7.2. Трошкови коришћења OpenAI API-ја 18
8. Имплементација веб решења *LegAI* 19
8.1. Опис апликације 19
8.2. Архитектура апликације 20
8.3. Имплементација *backend*-а апликације 20
8.3.1. Пројекат *AppAI.Api* 21
8.3.2. Пројекат *AppAI.Core* 24
8.3.3. Пројекат AppAI.Infrastructure 26
8.4. Имплементација *frontend*-а апликације 27

8.5. Случајеви коришћења 29
9. Приказ рада апликације 33
9.1. Почетна страница 33
9.2. Страница „*Šabloni*“ 33
9.3. Страница за регистрацију корисника 34
9.4 Страница за пријаву корисника 35
9.5. Кориснички профили 35
9.5.1. Профил корисника 35
9.5.2. Профил администратора 36
9.5.3. Измена личних података 37
9.6 Шаблони правних докумената 37
9.6.1. Генерисање шаблона правних докумената 37
9.6.2. Брисање постојећих шаблона 39
9.6.3. Претрага шаблона 40
9.6.4. AI препорука шаблона 42
9.6.5 Чаробњак за аутоматско генерисање докумената 43
9.7 Документи корисника 48
9.7.1. Историја корисникових докумената 48
9.7.2. Брисање генерисаних докумената 48
9.7.3 Претрага докумената корисника 49
9.8 AI асистент 51
9.9. Администраторски панел 52
10. Закључак 53
11. Референце 54

Веб решење за аутоматско генерисање и управљање правним документима уз асистенцију вештачке интелигенције

САЖЕТАК

Разумевање правне документације представља један од најчешћих изазова са којима се грађани сусрећу у свакодневном животу. Сложена терминологија, велики број различитих шаблона и честе формалне недоумице доводе до ниског нивоа правне писмености у популацији. Због тога је потребан систем који би поједноставио овај процес и учинио правне садржаје доступнијим широј јавности.

У овом раду представљено је веб решење *LegAI*, систем који комбинује савремене веб технологије са вештачком интелигенцијом ради аутоматског генерисања правних докумената. Систем користи интерактивни чаробњак („*wizard*“) за попуњавање шаблона, док *AI* модул омогућава корисницима да разумеју значење правних појмова, структуру документа и релевантне правне одредбе. На овај начин *LegAI* доприноси повећању правне писмености корисника и смањује ризик од грешака приликом израде важних докумената.

Техничка реализација обухвата *ASP.NET Core backend*, *React* кориснички интерфејс, *SQL Server* базу података и интеграцију *OpenAI API*-ја. *OpenXML SDK* коришћен је за генерисање *DOCX* докумената, док је посебан модул предвиђен за аутоматско претварање *DOCX* документа у *PDF*, што представља један од будућих праваца унапређења система.

LegAI представља практично и доступно решење које поједностављује израду правних докумената, унапређује разумевање правне терминологије и доприноси дигитализацији административних процеса.

Кључне речи: аутоматско генерисање докумената, правна писменост, веб апликација, вештачка интелигенција, *OpenAI, OpenXML, ASP.NET Core, React.*

Web-Based Solution for Automatic Generation and Management of Legal Documents with the Assistance of Artificial Intelligence

ABSTRACT

Understanding legal documentation represents one of the most common challenges that citizens encounter in everyday life. Complex terminology, a large variety of document templates, and frequent formal ambiguities contribute to the generally low level of legal literacy among the population. For this reason, there is a need for a system that simplifies the process of creating legal documents and makes legal content more accessible to a wider audience.

This thesis presents *LegAI*, a web-based solution that combines modern web technologies with artificial intelligence to enable automatic generation of legal documents. The system uses an interactive *wizard* to guide users through template completion, while the *AI* module helps them understand the meaning of legal terms, the structure of the document, and relevant legal provisions. In this way, *LegAI* contributes to improving users' legal literacy and reduces the risk of errors when preparing important documents.

The technical implementation includes an *ASP.NET Core backend*, a *React*-based user interface, a *SQL Server* database, and integration with the *OpenAI API*. The *OpenXML SDK* is used for generating *DOCX* documents, while an additional module is planned for automatic *DOCX*-to-*PDF* conversion, representing one of the system's potential future upgrades.

LegAI provides a practical and accessible solution that simplifies the creation of legal documents, enhances the understanding of legal terminology, and supports the digitalization of administrative processes.

Keywords: automated document generation, legal literacy, web application, artificial intelligence, *OpenAI*, *OpenXML*, *ASP.NET Core*, *React*.

1. Увод

Правна документација представља неопходан део многобројних административних, пословних и личних процеса. Међутим, велики део становништва сусреће се са значајним потешкоћама приликом разумевања правних термина, структуре докумената и процедура које морају да испоштују. Правна терминологија често је сложена, формалистичка и нејасна обичним корисницима, што доводи до честих грешака приликом попуњавања формулара, неправилно сачињених уговора или до одустајања од правних радњи због несигурности. Управо ови изазови показују колико је важно подићи ниво грађанске правне писмености и омогућити приступачније алате који овај процес поједностављују.

У таквом окружењу јавља се потреба за дигиталним системом који може да помогне корисницима да лакше разумеју правне појмове, да праве мање грешака и да брже дођу до исправних докумената. Аутоматизација у комбинацији са вештачком интелигенцијом представља идеалан приступ, јер омогућава генерисање садржаја прилагођеног кориснику, уз објашњења, препоруке и подршку.

Овај рад представља развој веб решења *LegAI* – система који омогућава аутоматско генерисање правних докумената кроз интерактивни чаробњак („*wizard*“) и интеграцију напредних *AI* модела. Систем је осмишљен тако да буде приступачан и корисницима без икаквог правног знања. *AI* модул омогућава објашњење неразумљивих правних одредби, анализу упита и препоруку најприкладнијег документа, чиме се директно подиже ниво правне писмености становништва.

Са техничке стране, *LegAI* је реализован као модерна веб апликација у *ASP.NET Core* оквиру, dok je frontend развијен у *React*-у. *Microsoft SQL Server* база података садржи податке о корисницима, документима и шаблонима, док се *OpenXML SDK* користи за генерисање *DOCX* докумената. Овако дизајниран систем обезбеђује стабилност, проширивост и могућност даљег развоја.

Циљ овог рада је да прикаже процес креирања *LegAI* система, функционалности које нуди, техничку архитектуру и потенцијал који има за унапређење приступачности правних садржаја. Посебан акценат ставља се на улогу вештачке интелигенције у подизању правне писмености, као и на могућностима аутоматизације које значајно поједностављују поступак израде правних докумената.

2. Архитектура и технологије веб решења

Развој савремених веб решења заснива се на избору адекватних технологија и јасно дефинисаној архитектури која омогућава стабилан, безбедан и ефикасан рад система. Веб решење ***LegAI*** реализовано је применом *ASP.NET* технологије за серверску логику (енг. *backend*) и *React* оквира за развој корисничког интерфејса (енг. *frontend*). За обезбеђивање перзистенције података, примењена је *Microsoft SQL Server* база података.

Microsoft SQL Server је релациона база података која обезбеђује трајно, сигурно и структурирано чување свих информација у систему. *SQL Server* користи *T-SQL* (*Transact-SQL*) као језик за рад са базама, омогућавајући извршавање упита, рад са различитим типовима података, као и подршку за трансакције и аналитичке операције. Ова платформа нуди поуздане перформансе, висок ниво безбедности података и добру усклађеност са алатима за обраду и анализу података. [1]

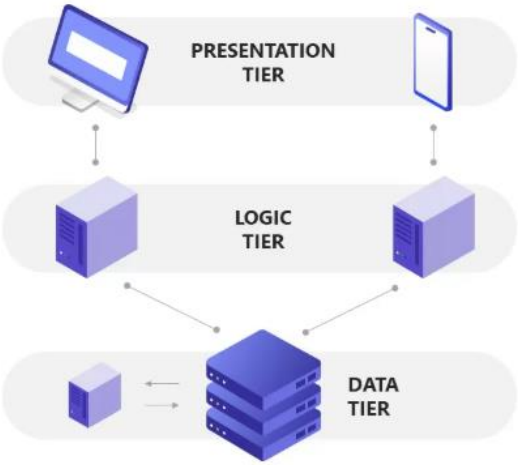
ASP.NET Core технологија представља један од водећих оквира за развој веб сервиса на *.NET* платформи, развијен од стране *Microsoft*-а. Овај оквир је мултиплатформски, што значи да се може користити на различитим оперативним системима, укључујући *Windows*, *Linux* и *macOS*. *.NET* је платформа за развој софтвера која обухвата различите алате, програмске језике и библиотеке, и служи за прављење најразличитијих врста апликација. [2]

React је *JavaScript* библиотека намењена развоју корисничких интерфејса, која омогућава креирање динамичких и интерактивних веб апликација коришћењем компонентно оријентисаног приступа. Уместо да се интерфејс гради као једна целина, *React* га дели на мање, самосталне компоненте које управљају сопственим стањем и могу се поново користити у различитим деловима апликације. Када дође до промене података, *React* ефикасно ажурира само оне компоненте чији се приказ мења, чиме се постижу боље перформансе и пријатније корисничко искуство. [3]

Посебна компонента *LegAI* система односи се на генерисање докумената, што је реализовано употребом ***OpenXML SDK*** библиотеке за креирање *Word* докумената. Ова технологија омогућава да се сви документи генеришу тачно, брзо и у формату који корисницима највише одговара. [4]

Укључена је и интеграција са ***OpenAI API* сервисом**, који омогућава употребу савремених модела вештачке интелигенције. *OpenAI API* функционише као засебан логички слој који обрађује текстуалне упите корисника и враћа прецизне, контекстуалне одговоре. [5]

На слици 2.1. приказана је архитектура веб апликације.



Слика 2.1. *Приказ архитектуре веб апликације.* [6]

2.1. Клијентски и серверски део апликације

Веб апликације се састоје из два главна дела: клијентског (енг. *frontend*) и серверског (енг. *backend*) дела. Клијентски део извршава се у веб претраживачу (енг. *web browser)* корисника као што су *Google Chrome, Mozilla Firefox, Microsoft Edge* или *Safari*, док се серверски део налази на удаљеном серверу који обрађује захтеве, приступа подацима и враћа одговарајуће одговоре. Иако обављају различите улоге, ова два дела функционишу као јединствен систем, стално комуницирајући како би кориснику обезбедили брз, стабилан и поуздан рад апликације.

Клијентски део апликације *LegAI* реализован је коришћењем *React* библиотеке, која омогућава развој динамичких веб интерфејса и брзо ажурирање садржаја без поновног учитавања странице. *React* користи компонентну архитектуру, што значи да је целокупни интерфејс подељен на мање, јасно дефинисане целине као што су: приказ шаблона, „*wizard*“ за генерисање докумената, *AI* панел, историја докумената и кориснички мени.

Клијентски део је одговоран за:

- визуелни приказ интерфејса,
- унос података током генерисања докумената,
- комуникацију са *AI* асистентом,
- преглед листе шаблона,
- претрагу и преглед генерисаних докумената.

Предности клијентског дела:

- Високе перформансе: Обрада се извршава на корисничком уређају, што омогућава брз одзив интерфејса.
- Интерактивност: *React* омогућава динамичке промене садржаја без поновног учитавања странице.
- Мање оптерећење сервера: Много логике обрађује се локално, а сервер прима само неопходне захтеве.
- Модуларност: Компоненте се могу лако надоградити или заменити без утицаја на остатак система.

Овакав приступ омогућава да *LegAI* пружи корисницима брзо и пријатно искуство, што је важно код апликација које раде са више сложених екрана и формулара.

Серверски део *LegAI* апликације развијен је коришћењем *ASP.NET Core* технологије, познате по стабилности, перформансама и подршци за велик број истовремених корисника. *Backend* представља „срце“ система, јер обрађује податке, управља логиком апликације, комуницира са базом података и позива спољне сервисе као што је *OpenAI API*.

Главне улоге серверског дела:

- Обрада захтева: Сваки захтев који стигне од клијента пролази кроз сервер и његову логику.
- Комуникација са базом: Чување и преузимање корисника, шаблона, докумената и *AI* историје.
- Валидација: Провера исправности података које корисник уноси.
- Генерисање докумената: Креирање *DOCX* фајлова (*OpenXML*).
- *AI* интеграција: Слање упита *OpenAI API*-ју и обрада добијених одговора.

Кључне карактеристике *backend*-а:

- Сигурност: Осетљиви подаци и логика налазе се на серверу, изоловани од јавности.
- Стабилност: *ASP.NET Core* омогућава поуздан рад чак и под већим оптерећењем.
- Скалабилност: Архитектура је погодна за даље ширење.
- Перформансе: Висока брзина обраде и оптимизација меморије.

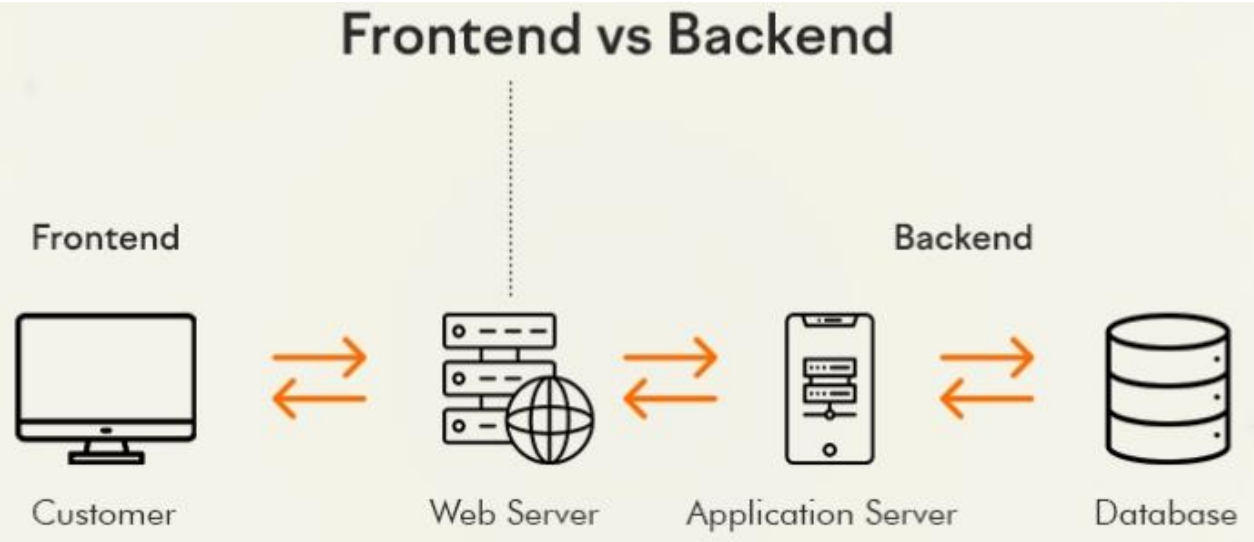
Ове карактеристике чине серверски део темељем целокупног система.

Клијентски и серверски део комуницирају преко *HTTP* протокола, размењујући податке у *JSON* формату. Пример комуникације:

1. Корисник покреће „*wizard*” за генерисање документа.
2. Клијент шаље податке серверу.
3. Сервер проверава податке, приступа бази и покреће процес генерисања документа.
4. Када је документ спреман, сервер шаље одговор са линком за преузимање.
5. Клијент приказује резултат кориснику.

Оваква комуникација омогућава да цео систем функционише брзо, стабилно и предвидиво — што је веома важно за професионалне веб апликације попут *LegAI*.

На слици 2.1.1. приказана је комуникација клијентског и серверског дела апликације.



Слика 2.1.1. *Приказ комуникације клијентске и серверске стране.*[7]

3. ASP.NET

ASP.NET представља савремени *open-source* веб оквир који је развила компанија *Microsoft*, и који служи за изградњу веб апликација, веб сервиса и *cloud* оријентисаних решења у оквиру *.NET* платформе. Као *cross-platform* технологија, *ASP.NET* може да се извршава на различитим оперативним системима, укључујући *Windows, Linux* и *macOS*, а такође подржава и *Docker* контејнеризацију.

Оквир проширује функционалности основне *.NET* платформе додавањем алата и библиотека намењених веб развоју. То подразумева механизме за руковање *HTTP* захтевима, рад у језику *C#* (или *F#*), као и подршку за популарне архитектонске обрасце као што је *Model-View-Controller* (*MVC*). Поред тога, *ASP.NET* нуди интегрисане механизме за аутентификацију и ауторизацију корисника, укључујући и напредни *multi-factor authentication*, што доприноси безбедности веб апликација.

У оквиру *ASP.NET*-а могуће је развијати различите типове система: од класичних динамичких веб страница, преко *REST API*-ја и микросервисних архитектура, па све до *real-time* функционалности путем *SignalR* технологије. *ASP.NET* је у потпуности отвореног кода и активно се развија у оквиру заједнице, уз хиљаде доприносиоца и компанија које учествују у његовом унапређењу. [8]

3.1. ASP.NET Core

ASP.NET Core је савремени, отворени и вишеплатформски оквир намењен развоју веб апликација које захтевају високе перформансе и поузданост. Он представља еволутивни корак у односу на старији *ASP.NET*, јер је у потпуности редизајниран тако да буде лакши, модуларнији и флексибилнији. Због тога се може извршавати на различитим оперативним системима, укључујући *Windows, Linux* и *macOS*, што програмерима омогућава да раде у окружењу које им највише одговара.

За разлику од монолитног приступа претходне генерације, *ASP.NET Core* користи концепт компоненти и *middleware*-а, што програмерима даје већу контролу над тим како се апликација понаша током обраде *HTTP* захтева. Ова архитектура омогућава креирање апликација различите намене — од класичних веб интерфејса, преко *REST API*-ја, па све до *cloud-native* микросервисних решења.

Поред тога, *ASP.NET Core* је прилагођен модерном web развоју и добро се интегрише са популарним *frontend* оквирима и алатима. То олакшава изградњу динамичких и интерактивних апликација које могу да се извршавају и локално и у *cloud* окружењима. Уграђени механизми као што су *dependency injection*, систем конфигурације и унифицирана обрада захтева доприносе једноставнијем развоју, тестирању и каснијем одржавању апликација.

Захваљујући активној заједници и *Microsoft*-овој подршци, *ASP.NET Core* се брзо развија и добија нове могућности, због чега је постао један од најпопуларнијих избора за развој модерних веб решења. [9]

3.1.1 ASP.NET Core Web API

ASP.NET Core Web API представља оквир који омогућава изградњу *REST* сервиса намењених комуникацији између клијента и сервера путем *HTTP* протокола. Архитектура се заснива на јасној подели улога: клијент иницира захтеве, док сервер прихвата те захтеве, обрађује их, приступа подацима и шаље одговоре назад. За разлику од класичних веб апликација, сервер у *Web API* моделу не генерише кориснички интерфејс, већ искључиво враћа податке у стандардизованим форматима као што су *JSON* или *XML*.

ASP.NET Core Web API следи *REST* принципе и користи уобичајене *HTTP* методе — ***GET***, ***POST***, ***PUT*** и ***DELETE*** — које одговарају основним *CRUD* операцијама. Овакав приступ програмерима омогућава да на једноставан начин креирају сервисе који могу да се интегришу са апликацијама на различитим платформама, укључујући мобилне апликације, веб клијенте или друге *backend* системе.

Кључни елементи овог модела су **модел** и **контролер**.

- **Модел** описује структуру података који се размењују у апликацији. Он представља основу за валидацију и манипулацију подацима.
- **Контролер** је задужен за прихватање *HTTP* захтева, позивање пословне логике и враћање одговора клијенту. Контролер најчешће шаље податке у *JSON* формату, а поред тога може да управља процесима као што су ауторизација, аутентификација и обрада грешака.

Коришћење *Web API* приступа доноси високу флексибилност, јер се серверска страна може развијати, ажурирати или скалирати независно од клијента. Пошто се комуникација одвија путем стандардног *HTTP* протокола, интеграција са другим системима је поуздана и једноставна.

3.1.2. Сигурност и аутентификација у ASP.NET Core-у

Сигурност је један од најважнијих аспеката сваке веб апликације, а *ASP.NET Core* нуди развијен систем за аутентификацију, ауторизацију и управљање корисницима. Ови механизми обезбеђују да осетљивим подацима и заштићеним ресурсима могу да приступе само корисници који имају одговарајуће привилегије.

Аутентификација представља процес идентификације корисника. У *ASP.NET Core*-у се спроводи преко *middleware* компоненти, које омогућавају више типова аутентификације, укључујући:

- аутентификацију засновану на колачићима,
- коришћење *JSON Web Token-а* (*JWT*),
- интеграцију са спољним провајдерима као што су *Google*, *Facebook* и *Microsoft*.

Овај систем је дизајниран тако да омогућава лаку конфигурацију и прилагођавање, што програмерима даје могућност да изаберу стратегију која најбоље одговара специфичним потребама пројекта. [10]

Ауторизација дефинише ниво приступа корисника одређеним ресурсима или функционалностима. У *ASP.NET Core*-у може да се примени на нивоу читаве апликације, на појединачним контролерима или на специфичним акцијама.

Ова флексибилност омогућава детаљну контролу приступа, засновану на корисничким улогама, правилима или прилагођеним политикама [11].

Поред основних механизама аутентификације и ауторизације, *ASP.NET Core* нуди и моћан систем за управљање корисницима — ***ASP.NET Core Identity***. Овај оквир обезбеђује функционалности као што су регистрација, пријављивање, управљање улогама, ресетовање лозинке и двофакторска аутентификација (*2FA*). *Identity* је флексибилан и може се прилагодити различитим архитектурама — од једноставних апликација до сложених корпоративних система. [12]

Добро организовано управљање корисницима и контролом приступа значајно повећава безбедност апликације, смањује могућност злоупотреба и доприноси стабилности читавог система.

3.2. ASP.NET Web API

Web API представља механизам који омогућава различитим софтверским системима да размењују податке коришћењем *HTTP* протокола. У пракси, *API (Application Programming Interface)* функционише као „договорени интерфејс“ који дефинише на који начин клијент може да приступи одређеним функцијама или подацима на серверу. Кроз стандардизоване методе и формате података, *API* омогућава апликацијама да комуницирају без обзира на то на којој платформи или у ком програмском језику су развијене.

ASP.NET Web API је оквир у *.NET* екосистему који омогућава изградњу сервиса доступних путем *HTTP*-а. Он је намењен прављењу *API*-ја које могу да користе веб апликације, мобилне апликације, игре, *IoT* уређаји и други системи. Иако је концептуално сличан *ASP.NET MVC* платформама, *Web API* се фокусира на слање података у машински читљивом облику (нпр. *JSON*), уместо *HTML* садржаја.

Главне карактеристике *ASP.NET Web API*-ја:

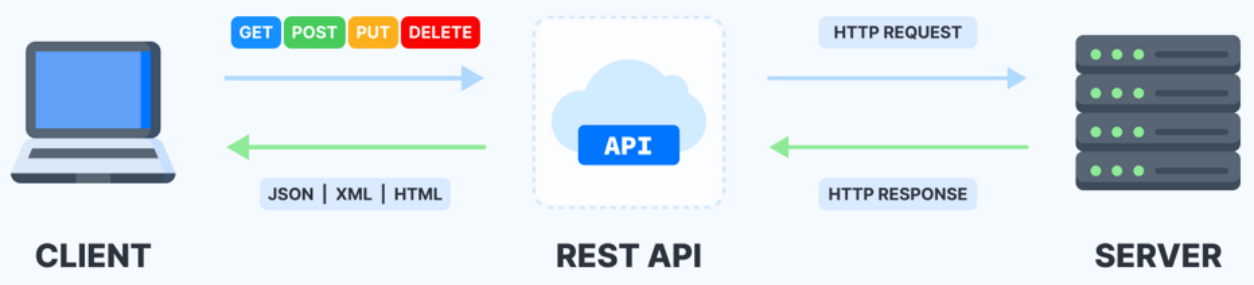
- погодан за развој *RESTful* сервиса;
- изграђен на *ASP.NET* инфраструктури, што омогућава приступ постојећем *request/response* механизму;
- *HTTP* методе се директно повезују са методама контролера;
- омогућава размену података у више формата (*JSON, XML, BSON...*);
- може да се хостује у *IIS*-у, али и као самостална апликација;
- *HttpClient* библиотека омогућава једноставно слање захтева ка *API*-ју из различитих типова апликација.

Због своје флексибилности, *Web API* се често користи за развој *cloud* сервиса, *backend* решења за *SPA* апликације и комуникацију са мобилним уређајима. [13]

3.2.1. REST Web

REST API (*Representational State Transfer Application Programming Interface*) је архитектонски стил који дефинише начин на који се веб сервиси организују и како клијенти приступају ресурсима путем *HTTP*-а. Основна идеја *REST*-а је да се подаци третирају као „ресурси“ који се налазе на одређеним *URI* адресама. Клијент шаље захтев, сервер га обрађује и враћа репрезентацију ресурса, обично у *JSON* формату.

На слици 3.2.1. приказана је употреба *RESTful API*-а.



Слика 3.2.1. *Приказ употребе RESTful API-а.*[14]

- **Основни принципи *REST*-а**

  1. ***Stateless*:**

     Сервер не чува стање клијента. Сваки захтев носи све информације које су потребне да би сервер могао да одговори. То олакшава скалирање и распоређивање више инстанци сервера.

  2. **Ресурси као централни део система:**

     Пример апликације за управљање паметним уређајима у домаћинству:

     - /devices → листа свих уређаја;
     - /devices/{id} → детаљи једног уређаја;
     - /devices/{id}/status → тренутно стање уређаја (укључен/искључен).

  3. **HTTP методе као алати за манипулацију ресурсима:**

     - ***GET*** — читање података (нпр. дохватања температуре са термостата)
     - ***POST*** — додавање новог ресурса (нпр. регистрација новог паметног уређаја у систему)
     - ***PUT*** — ажурирање постојећег ресурса (нпр. промена назива уређаја)
     - ***DELETE*** — брисање ресурса (нпр. уклањање уређаја из домаћинске мреже)

4. **Репрезентација ресурса:**

   Ресурс може бити представљен најчешће као *JSON*, *XML* или други формати. Клијент дефинише шта жели путем *HTTP* заглавља.

- **Кључне компоненте *REST API*-ја:**

1. ***Endpoint*-и**

   То су *URI* адресе на које клијент шаље захтеве. На пример у систему за резервацију спортских термина:

   - **GET /api/users :** Списак свих корисника.
   - **GET /api/users/{id} :** Преузимање података за одређеног корисника.
   - **POST /api/users :** Креирање новог корисника слањем *JSON* објекта са детаљима.
   - **PUT /api/users/{id} :** Ажурирање података постојећег корисника.
   - **DELETE /api/users/{id} :** Брисање одређеног корисника.

2. **Одговори сервера*:***

   HTTP статус кодови описују шта се десило:

   - *200 OK* — успешна операција,
   - *201 Created* — креиран нови ресурс,
   - *403 Forbidden* — корисник нема дозволу,
   - *404 Not Found* — ресурс не постоји,
   - *500 Internal Server Error* — серверска грешка.

3. **Формати података:**

   У пракси се најчешће користи *JSON* јер је лаган и брзо се обрађује у веб и мобилним апликацијама.

- **Предности *REST API*-ја:**

1. **Висока интероперабилност** – ради на свим платформама које подржавају *HTTP*.
2. **Скалабилност** – *stateless* приступ омогућава рад са великим бројем корисника.
3. **Једноставност примене** – структура *REST* сервиса је логична и лако разумљива.
4. **Погодност за мобилне уређаје** – мали пакети података и брза размена.

4. React

React је популарна *open-source JavaScript* библиотека коју развија *Meta*, намењена изградњи динамичких и интерактивних корисничких интерфејса. Иако није потпун *framework*, захваљујући великом екосистему може да се користи за развој *single-page*, мобилних (*React Native*) и серверски рендерованих апликација. Кључну улогу у перформансама има *Virtual DOM*, који минимизује директне измене у стварном *DOM*-у и омогућава брзо ажурирање интерфејса.

React користи *JSX* синтаксу која омогућава комбиновање логике и *HTML* структуре у компонентама, што олакшава организацију и разумевање кода. Апликације се могу писати и у *JavaScript*-у и у *TypeScript*-у. [3]

4.1. Компоненте

Компоненте су основа *React* апликације. Свака компонента представља део интерфејса и садржи сопствену логику и приказ. Најчешће се користе функционалне компоненте, док су класне компоненте мање заступљене у модерном *React*-у.

Комуникација између компоненти иде преко **props-а**, док локално стање управља динамиком интерфејса. Оваква архитектура омогућава поновну употребу и добру организацију кода.

4.2. JSX и декларативни приказ

React се ослања на декларативан приступ. Уместо ручног управљања *DOM*-ом, програмер описује како интерфејс треба да изгледа, а *React* аутоматски примењује све измене. *JSX* омогућава да се *UI* структуре пишу читљиво и логично, уз директну везу са стањем компоненте.

4.3. Hooks

Hooks су један од најважнијих делова модерног *React*-а и омогућавају функционалним компонентама да користе стање и животни циклус.

Најважнији *hooks* су:

- ***useState*** – управљање локалним стањем,
- ***useEffect*** – асинхрони позиви и споредни ефекти,
- ***useContext*** – дељење података између компоненти.

Hook механизам смањује сложеност и омогућава лакшу употребљивост логике кроз *custom hooks*.

4.4. Управљање стањем

За веће апликације потребно је глобално стање. *React* нуди више решења:

- ***Context API*** – мањи обим дељеног стања,
- ***Redux*** – централизовано управљање комплексним стањем,
- ***Zustand / Recoil*** – новије и једноставније библиотеке,
- ***React Query*** – рад са серверским подацима, кеширање и синхронизација.

Правилан избор начина управљања стањем значајно утиче на перформансе и структуру апликације.

4.5. Рутирање

За навигацију у *SPA* (*Single Page Application*) апликацијама најчешће се користи ***React Router***, који омогућава дефинисање рута, рад са параметрима и креирање заштићених делова апликације. Рутирање функционише без поновног учитавања странице, што доприноси брзини и бољем корисничком искуству.

4.6. Рад са сервером

React се ослања на *fetch, async/await* и библиотеке као што су *Axios* или *React Query* за рад са *API*-јима. *React Query* је посебно користан за рад са серверским стањем, јер аутоматски кешира, обнавља и синхронизује податке.

4.7. React екосистем

React има један од најбогатијих екосистема у веб развоју. Најважније технологије које га прате су:

- ***Next.js*** – серверско рендеровање и *full-stack* развој,
- ***React Native*** – мобилне апликације,
- ***Material UI / TailwindCSS*** – *UI* компоненте и стилизовање.

Због ове флексибилности, *React* се користи у широком спектру апликација, од мањих веб интерфејса до сложених корпоративних решења.

5. SQL Server

Microsoft SQL Server (*MS SQL Server*) представља релациони систем за управљање базама података (*RDBMS*) који је развила компанија *Microsoft*, са циљем да обезбеди ефикасно складиштење, организовање и обраду података у различитим типовима софтверских система. Базе података у *SQL Server*-у се организују у табеле које се међусобно повезују, а свака табела садржи редове и колоне (атрибуте). Ова организациона структура омогућава систематско складиштење података и брзо преузимање информација. [15]

Једна од најважнијих карактеристика *SQL Server*-а је подршка за *ACID* принципе (*Atomicity, Consistency, Isolation, Durability*), што обезбеђује поузданост трансакција, чак и у случају прекида система или конкуренције више корисника. Поред тога, *SQL Server* нуди механизме за репликацију, клонирање података, високу доступност и аналитичку обраду у реалном времену, што га чини погодним и за мале апликације и за велике корпоративне системе.

SQL Server се лако интегрише са савременим развојним окружењима, посебно са *.NET* платформом. Кроз *ADO.NET, Entity Framework* и друге библиотеке, омогућено је креирање апликација које раде са подацима на објектно-оријентисан начин. Поред *.NET* технологија, *SQL Server* је компатибилан и са *Python*-ом, *Java*-ом, *JavaScript*-ом, као и са *cloud* сервисима као што су *Azure SQL Database* и *SQL Managed Instance*.

5.1. Кључне карактеристике SQL Server-а

1. Релациони модел података

SQL Server користи класични релациони модел у коме су подаци смештени у табелама повезаним примарним и страним кључевима. Могуће је дефинисати индексе, видове, тригере и процедуре које доприносе бољој организацији и ефикасности. Нормализација базе доприноси елиминацији вишка података и побољшава конзистентност.

2. Скалабилност и висока доступност

Систем подржава вертикално и хоризонтално скалирање, што га чини прилагодљивим за различите сценарије оптерећења. Технологије као што су *Always On Availability Groups, Failover Clustering* и *Database Mirroring* омогућавају да апликације наставе да раде чак и у случају квара, уз минималан губитак података.

3. Сигурност и управљање приступом

SQL Server има робустан безбедносни модел који укључује:

- **аутентификацију корисника** (*Windows* и *SQL login*),
- **ауторизацију** на нивоу база, табела и појединачних колона,
- **енкрипцију података** (*TDE — Transparent Data Encryption*),
- **аудит логове** за праћење активности.

Ови механизми чине *SQL Server* поузданим избором за апликације које раде са осетљивим подацима.

4. Интеграција са *.NET* и другим платформама

Захваљујући богатом сету алата, *SQL Server* се једноставно интегрише са различитим програмским језицима. За *.NET* постоје:

- ***ADO.NET*** за директну комуникацију са базом,
- ***Entity Framework Core*** за објектно-релационо мапирање (*ORM*),
- ***Dapper*** као лагана алтернатива за брзе упите.

SQL Server подржава и *REST API* приступ путем *Azure SQL*-а, што га чини флексибилним у *cloud* окружењима.

5.2. SQL и T-SQL

SQL (*Structured Query Language*) је језик који се користи за манипулацију релационим базама података. Његова примена обухвата креирање структура базе, дефинисање односа, рад са подацима, контролу приступа и одржавање базе.

Microsoft SQL Server користи своју проширену верзију *SQL* језика под називом ***Transact-SQL* (*T-SQL*)**. *T-SQL* уводи додатне команде, контролне структуре и функције које нису део стандардног *SQL*-а, што омогућава развијање комплексних логичких операција директно унутар базе.

T-SQL упити се могу извршавати кроз:

- ***SQL Server Management Studio* (*SSMS*)** — најчешће коришћени алат за рад,
- ***sqlcmd*** *CLI* алат,
- ***Azure Query Editor*** за *cloud* базе.

Основне *T-SQL* операције:

- ***INSERT*** — додавање нових записа у табелу,
- ***UPDATE*** — измена постојећих записа,
- ***DELETE*** — брисање редова који испуњавају услов,
- ***SELECT*** — упит за читање и приказивање података на основу критеријума.

Поред ових операција, *T-SQL* подржава *stored* procedure, функције, тригере и курсоре за сложеније сценарије управљања подацима.

6. OpenXML

У оквиру савремених *.NET* апликација све је чешћа потреба за потпуно аутоматизованим креирањем, обрадом и модификацијом докумената различитих типова – попут извештаја, уговора, пуномоћја, фактура или специјализованих образаца који се попуњавају подацима из базе. Једна од најзаступљенијих технологија за ову намену је ***OpenXML***, стандард који омогућава програмирање *Office* докумената без ослањања на *Microsoft Office* инсталацију на серверу или клијенту.

OpenXML представља отворени *Microsoft*-ов формат заснован на комбинацији *ZIP* контејнера и специјализованих *XML* датотека које описују структуру, садржај и стил документа. У пракси, сваки *.docx, .xlsx* или *.pptx* документ је организован као архивирана мапа са више *XML* чворова – који дефинишу текст, табеле, стилове, релације између делова документа, мултимедијалне ресурсе и друге елементе. Захваљујући томе, програмери могу да приступају овим деловима документа на ниском нивоу и прецизно контролишу како ће документ бити формиран. [4]

У *.NET* окружењу за рад са *OpenXML* технологијом користи се библиотека ***DocumentFormat.OpenXml***, која омогућава:

- креирање, читање и измену *.docx*, *.xlsx* и *.pptx* докумената без потребе за *Office COM* компонентама,
- пуњење унапред дефинисаних шаблона подацима добијеним из апликације или базе података,
- аутоматско генерисање табела, параграфа, наслова, заглавља, фуснота и стилских елемената,
- управљање сликама, графиконима и другим уграђеним ресурсима,
- контролу над структуром документа на нивоу *XML* чворова, што омогућава високу флексибилност у изградњи комплексних формата.

OpenXML се посебно истиче у ситуацијама када је потребно кориснику испоручити документ у *Office* формату који касније може ручно да уређује. То укључује, на пример, креирање правних докумената, извештаја, потврда и *Excel* табела са резултатима обраде података. Главна предност је у томе што коначни резултат изгледа као да је ручно креиран у *Word*-у или *Excel*-у, док је у стварности генерисан у потпуности аутоматизовано, директно из кода апликације.

Поред тога, *OpenXML* је стабилан и дугогодишњи стандард, званично подржан од стране *Microsoft*-а, што га чини поузданим избором за системе који генеришу велике количине докумената или морају да раде у окружењу у коме није дозвољено ослањање на *Office* инсталацију, као што су *cloud* сервери, контејнерске апликације или аутоматизоване *CI/CD* платформе.

7. OpenAI

OpenAI је истраживачка организација и компанија која се бави развојем напредних модела вештачке интелигенције. Основана је 2015. године са идејом да *AI (Artificial Intelligence)* технологије постану корисне, доступне и безбедне за широк спектар примена. Највећи део њиховог рада усмерен је на развој модела који могу да разумеју и генеришу природан језик, решавају комплексне задатке и аутоматизују процесе који би иначе захтевали људску интелигенцију.

Најпознатији *OpenAI* модели припадају *GPT* серији (*Generative Pre-trained Transformer*). Ови модели користе трансформер архитектуру и могу да раде све — од писања текста, превођења и сумирања, до анализе садржаја и генерисања идеја. Због велике количине података на којима су тренирани, *GPT* модели производе одговоре који су логични, повезани и стилистички природни.

OpenAI омогућава коришћење ових модела кроз свој *API*, што програмерима даје могућност да *AI* функционалности укључе у сопствене апликације. Преко *API*-ја могуће је реализовати:

- генерисање текста и аутоматско писање,
- анализа садржаја и класификација,
- рад са сликама,
- препознавање и синтеза говора,
- препоруке, предлози или интерактивни асистенти.

Важан део *OpenAI*-а је и безбедност. Њихова платформа долази са смерницама и правилима коришћења која имају за циљ да се технологија употребљава одговорно и у складу са етичким стандардима.

7.1. Интеграција OpenAI API-ја у .NET апликације

Интеграција *OpenAI*-а у *.NET* апликације заснива се на слању *HTTP* захтева ка њиховом *API*-ју. Типичан сценарио подразумева *ASP.NET Core Web API* који служи као клијент. Основни кораци:

1. **Чување API кључа** - *API* кључ се чува у *appsettings.json* или као променљива окружења, како не би био у коду.
2. **Конфигурисање HttpClient-а** - У *.NET*-у се најчешће користи *HttpClient* регистрован преко *IHttpClientFactory*. Ту се поставља основни *URL* за *OpenAI* и *Bearer* токен.
3. **Слање упита моделу** - У посебној сервис класи формира се *JSON* тело захтева — модел који се користи, текст *prompt*-а и параметри. Након слања *POST* захтева, одговор стиже у *JSON* формату и десеријализује се у *.NET* објекат.
4. **Коришћење добијених података** - Резултати се могу приказати кориснику, снимити у базу, обрадити или комбиновати са другим подацима.

Овај приступ се може применити и у конзолним апликацијама, десктоп алатима или позадинским сервисима. Због тога се *OpenAI* лако интегрише у различите *.NET* архитектуре и додаје пројекту интелигентне и динамичке могућности.

7.2. Трошкови коришћења OpenAI API-ја

У овом пројекту коришћен је приступ *OpenAI* моделима путем *API*-ја, што подразумева систем наплате заснован на потрошњи. Да би *API* био активан, корисник треба да уплати почетни кредит, а затим се сваки захтев који апликација пошаље *OpenAI* серверу наплаћује у складу са бројем потршених токена. Токен представља најмању јединицу текста коју модел обрађује, и укупан трошак зависи од количине улазног текста и дужине генерисаног одговора. Иако се не ради о класичној месечној претплати, корисник уплаћује одређен износ кредита који се постепено троши током рада апликације. Када се расположиви кредит потроши, *API* аутоматски престаје са радом, што кориснику омогућава јасан увид у потрошњу и спречава настајање нежељених трошкова.

У оквиру веб решења реализованог у овом раду коришћен је модел ***GPT-4.1-mini***, који припада најповољнијој категорији савремених *OpenAI* модела. Овај модел обезбеђује добру равнотежу између цене и перформанси, што га чини практичним избором за апликације које обрађују већи број захтева. Преглед цена за водеће моделе представљен је у Табели 1, на основу званичних података доступних у новембру 2025. године. [16] [17]

Табела 1. *Трошкови коришћења OpenAI API модела у новембру 2025. године*

Модел	**Цена по 1.000.000 улазних токена (input)**	**Цена по 1.000.000 излазних токена (output)**	**Опис**
GPT-5.1	1.25 USD	10.00 USD	Најновији модел са највише способности за сложене задатке
GPT-5 mini	0.25 USD	2.00 USD	Буџетска, јефтинија верзија, погодна за чешће позиве API-ја
GPT-5 nano	0.05 USD	0.40 USD	Најјефтинија опција - корисна за најједноставније задатке
GPT-4.1	2.00 USD	8.00 USD	Компромис између цене и могућности
GPT-4.1-mini	0.40 USD	1.60 USD	Веома повољан за честе текстуалне упите
GPT-4.1 nano	0.10 USD	0.40 USD	Минимални модел — за једноставне, лаке захтеве

8. Имплементација веб решења *LegAI*

У овом поглављу описано је веб решење *LegAI*, које је намењено аутоматском генерисању и управљању правним документима уз помоћ вештачке интелигенције. Поглавље представља практичну реализацију свих технологија које су описане у претходним деловима рада – *ASP.NET Core* за *backend*, *ReactJS* за *frontend*, *Microsoft SQL Server* за базу података и *OpenAI API* за интеграцију интелигентних функционалности.

8.1. Опис апликације

LegAI је веб решење дизајнирано да омогући корисницима брзо, једноставно и поуздано креирање различитих правних докумената. Концепт је заснован на коришћењу унапред дефинисаних правних шаблона, који се попуњавају кроз интерактивни „*wizard*“ – систем који води корисника корак по корак кроз попуњавање свих потребних поља.

Након што корисник уноси потребне податке, *backend* спаја шаблон и вредности које је корисник унео и аутоматски генерише ***DOCX*** документе спремне за преузимање. Сви креирани документи чувају се у историји корисника, како би касније могли поново да се преузму или употребе.

Апликација подржава две корисничке улоге:

1. **Регистровани корисник,** који има приступ следећим функционалностима:
    - креирање налога, пријава и аутентификација,
    - преглед и уређивање профила,
    - преглед листе доступних шаблона,
    - попуњавање *wizard* форме и генерисање документа,
    - преузимање *DOCX* фајлова,
    - историја докумената,
    - претрага и филтрирање докумената,
    - AI асистент:
        - објашњење правних појмова и клаузула путем чета,
        - препорука шаблона на основу описане ситуације.
2. **Администратор** има све што има регистровани корисник, али уз додатне овлашћене функције:
    - креирање нових шаблона (назив, категорија, опис, *.docx* документ),
    - брисање постојећих шаблона,
    - увид у статистике и листу корисника.

Улога администратора представља „правника“ у систему – он дефинише правни текст шаблона који корисници затим користе за генерисање докумената.

Апликација је креирана тако да буде јасна, брза и интуитивна, а уз *AI* подршку корисницима пружа лакше разумевање правних термина и професионалних клаузула.

8.2. Архитектура апликације

Архитектура *LegAI* веб решења заснована је на раздвојеној структури *frontend*-а и *backend*-а, што омогућава независан развој, лако одржавање и добру организацију кода. Систем се састоји из три главне целине:

Frontend – ReactJS

Frontend је реализован као *SPA* (енг. *Single Page Application*) коришћењем *React* библиотеке. Овај део апликације омогућава:

- брзу навигацију без поновног учитавања странице,
- динамичко генерисање форми на основу шаблона добијених са *backend*-а,
- аутентификацију путем *JWT* токена,
- интеграцију са *AI* функцијама,
- приказ генерисаних докумената и управљање историјом.

React структура укључује компоненте, сервисе за комуникацију са *backend*-ом, контекст за управљање пријављеним корисником и посебне странице за све главне функционалности.

Backend – ASP.NET Core Web API

Backend представља централни део система и задужен је за:

- аутентификацију корисника (*JWT*),
- рад са шаблонима и генерисање докумената,
- комуникацију са *OpenAI* сервисом,
- чување и приступ подацима у *SQL Server*-у.

Backend је организован у више пројеката који раздвајају ентитете, сервисе и контролере. Оваква структура обезбеђује јасну архитектуру и бољу одрживост система.

База података – *Microsoft SQL Server*

SQL Server чува:

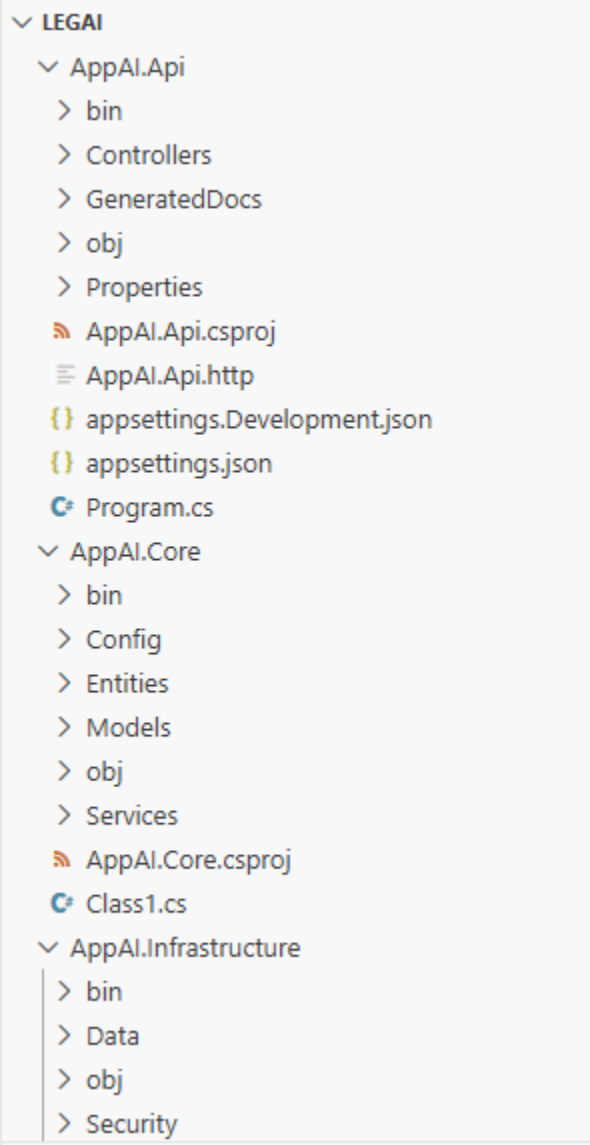
- податке о корисницима,
- правне шаблоне,
- све генерисане документе,
- информације о улогама, категоријама и параметрима шаблона.

Ентитети су оформљени помоћу *Entity Framework Core ORM*-а, што омогућава једноставну манипулацију подацима.

8.3. Имплементација *backend*-а апликације

Backend апликације *LegAI* развијен је у оквиру *ASP.NET Core* платформе и организован је као вишеслојни систем. Циљ овакве организације је да се јасно раздвоје *API* слој, пословна логика и модели домена, и инфраструктурни слој задужен за приступ бази података и интеграцију са спољним сервисима (*OpenAI*).

Решење се састоји из три главна пројекта: *AppAI.Api, AppAI.Core, AppAI.Infrastructure.* На слици 8.3.1. је приказана структура задње стране веб решења *LegAI*.



Слика 8.3.1. *Приказ структуре задње стране веб решења LegAI.*

У наставку су најважније компоненте и фајлови апликације.

8.3.1. Пројекат *AppAI.Api*

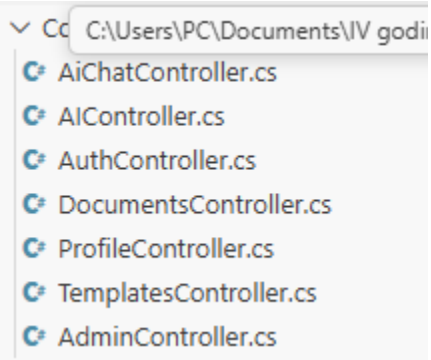
Пројекат *AppAI.Api* представља улазну тачку *backend*-а. У њему се налазе контролери и конфигурација *ASP.NET Core* апликације. Овај слој не садржи пословну логику – његова улога је да прими *HTTP* захтев, проследи га сервисима из доњих слојева и врати одговор клијенту.

• *Controllers*

Директоријум *Controllers* у апликацији представља важан слој задужен за обраду корисничких захтева и дефинисање *API* крајњих тачака (енг. *endpoints*) које омогућавају

комуникацију између клијента и сервера. У њему се налазе контролери који служе као веза између клијентске и серверске логике. Њихов задатак је да прихвате *HTTP* захтеве као што су *GET*, *POST*, *PUT* или *DELETE*, обраде их и пошаљу одговарајући одговор назад клијенту.

На слици 8.3.1.1. је приказана структура *Controllers* фолдера.



Слика 8.3.1.1. *Приказ садржаја Controllers фолдера.*

• ***GeneratedDocs***

Фолдер *GeneratedDocs* служи као радни директоријум за све фајлове које систем генерише током рада. Овај фолдер је везан за физичко чување докумената на диску.

• ***Properties***

Директоријум *Properties* садржи конфигурационе датотеке специфичне за развојно окружење. Најважнија међу њима је *launchSettings.json*, која омогућава коришћење специфичних *URL*–ова, извор профила за покретање, подешавање окружења, као и друге потребне параметре за локално тестирање апликације. Ове поставке не утичу на пословну логику, али су кључне за покретање и тестирање *API*-ја током развоја. На слици 8.3.1.2. је приказан садржај фајла *launchSettings.json*.

```
{
  "$schema": "http://json.schemastore.org/launchsettings.json",
  "iisSettings": {
    "windowsAuthentication": false,
    "anonymousAuthentication": true,
    "iisExpress": {
      "applicationUrl": "http://localhost:59419",
      "sslPort": 44339
    }
  },
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "http://localhost:5188",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "https://localhost:7264;http://localhost:5188",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "IIS Express": {
      "commandName": "IISExpress",
      "launchBrowser": true,
      "launchUrl": "swagger",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

Слика 8.3.1.2. *Приказ садржаја фајла launchSettings.json.*

• ***Program.cs***

Датотека *Program.cs* је главна улазна тачка *ASP.NET Core* апликације. Користи се као основа за покретање апликације и интеграцију различитих компоненти. Садржи:

- регистрацију контролера и *middleware*-а,
- повезивање са *SQL Server базом* (*AppDbContext*),
- конфигурацију *JWT* аутентификације,
- повезивање *OpenAI* подешавања (читање *OpenAISettings* из конфигурације),
- подешавање *CORS* правила за комуникацију са *React* клијентом.

• ***appsettings.json***

Датотека *appsettings.json* представља конфигурацију у *JSON* формату. У њој се налазе важне конфигурације као што су подаци за повезивање са базом података, параметри за *JWT*, конфигурација за *OpenAI*, подешавања логовања и друга глобална подешавања.

На слици 8.3.1.3. је приказан садржај фајла *appsettings.json.*

```
{
  "ConnectionStrings": {
  "DefaultConnection": "Server=(localdb)\\MSSQLLocalDB;Database=AppAIDb;Trusted_Connection=True;MultipleActiveResultSets=true"
  },
  "JwtSettings": {
    "SecretKey": "SUPERTAJNI_KLJUC_PROMENI_ME_123456789",
    "Issuer": "AppAI",
    "Audience": "AppAIClient",
    "ExpiryMinutes": 60
  },
    "OpenAI": {
    "ApiKey": "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",
    "Model": "gpt-4.1-mini"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Слика 8.3.1.3. *Приказ садржаја фајла appsettings.json*

8.3.2. Пројекат *AppAI.Core*

Пројекат ***AppAI.Core*** садржи доменску логику *LegAI* система – ентитете, *DTO* моделе и интерфејсе сервиса. Овај слој не зависи од конкретне базе података или спољних сервиса и представља “срце” апликације.

- ***Config***

Фолдер *Config* садржи конфигурационе класе које се користе за типизовано везивање подешавања из *appsettings* датотека. Типични примери су:

  - *JwtSettings.cs* – чување вредности везаних за генерисање и верификацију *JWT* токена,
  - *OpenAISettings.cs* – подаци потребни за комуникацију са *OpenAI* платформом.

Ове класе омогућавају да се конфигурација не користи као “обичан *string*”, већ као типизовани објекат. На слици 8.3.1.4. је приказан садржај фајла *JwtSettings.cs.*

```
namespace AppAI.Core.Config;

4 references
public class JwtSettings
{
    2 references
    public string SecretKey { get; set; } = null!;
    2 references
    public string Issuer { get; set; } = "AppAI";
    2 references
    public string Audience { get; set; } = "AppAIClient";
    1 reference
    public int ExpiryMinutes { get; set; } = 60;
}
```

Слика 8.3.1.4. *Приказ садржаја фајла JwtSettings.cs*

На слици 8.3.1.5. је приказан садржај фајла *OpenAISettings.cs*

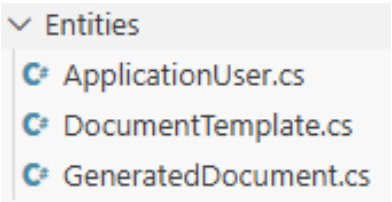
```
namespace AppAI.Core.Config;

3 references
public class OpenAISettings
{
    2 references
    public string ApiKey { get; set; } = null!;
    1 reference
    public string Model { get; set; } = "gpt-4.1-mini";
}
```

Слика 8.3.1.5. *Приказ садржаја фајла OpenAISettings.cs*

- ***Entities***

Фолдер *Entities* садржи ентитете који описују главне појмове домена: корисника, шаблон документа и генерисан документ. Сваки ентитет одговара једној табели у бази података и садржи својства која представљају колоне табеле. Користи се у оквиру *AppDbContext* класе у инфраструктурном слоју.

Ентитети представљају "стварне" објекте система и над њима се надовезује сва пословна логика. На слици 8.3.1.6. је приказана структура *Entities* фолдера.



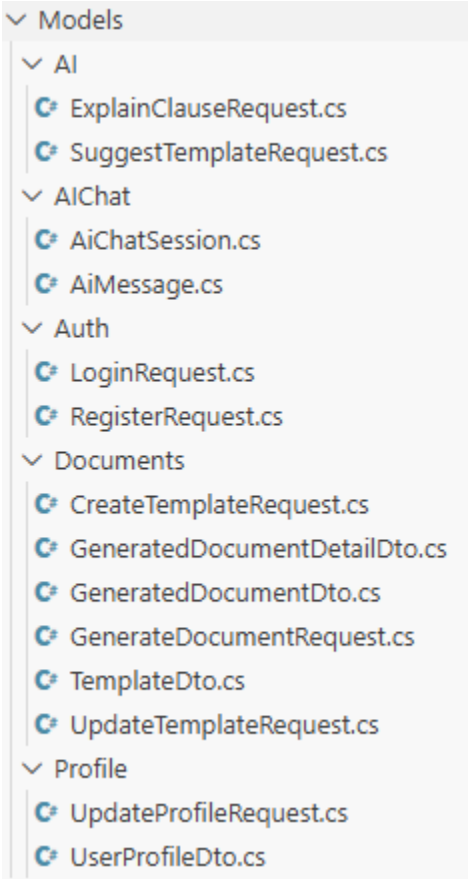
Слика 8.3.1.6. *Приказ садржаја Entities фолдера.*

- ***Models***

Фолдер *Models* садржи *DTO* (*Data Transfer Object*) моделе, који представљају објекте за пренос података. *DTO* модели се најчешће примењују у контролерима. Ови модели служе искључиво за размену података између слојева (*controller* ↔ *service* ↔ *frontend*), омогућавају пренос само неопходних података и не садрже пословну логику. Организовани су по логичким целинама:

- *AI* – модели захтева који се шаљу *OpenAI* сервису (нпр. опис ситуације за препоруку шаблона, текст клаузуле за објашњење),
- *AIChat* – модели који описују структуру *AI chat* сесије и појединачних порука,
- *Auth* – модели за регистрацију и пријаву корисника,
- *Documents* – модели за креирање, приказ и филтрирање докумената и шаблона,
- *Profile* – модели за приказ и измену података профила корисника.

На слици 8.3.1.7. је приказана структура *Models* фолдера.



Слика 8.3.1.7. *Приказ садржаја Models фолдера.*

8.3.3. Пројекат AppAI.Infrastructure

Пројекат *AppAI.Infrastructure* представља инфраструктурни слој апликације. Његова улога је да повеже доменску логику из *Core* слоја са конкретним технологијама – базом података и *OpenAI* сервисом.

• ***Data***

Фолдер *Data* садржи фајл *AppDbContext.cs* и фолдер *Migrations*. Фајл *AppDbContext.cs* је *Entity Framework Core* контекст који дефинише *DbSet* ентитете и конфигурацију базе. *Migrations* су аутоматски генерисане миграције које креирају и ажурирају структуру *SQL Server* базе. Све операције над базом (читање, упис, измена, брисање) пролазе кроз овај контекст.

• ***Services***

Фолдер *Services* садржи конкретне имплементације сервисног слоја. Најважнија класа у овом фолдеру је *OpenAIService*. Ова класа чита подешавања из *OpenAISettings*, припрема захтеве ка *OpenAI API*-ју, шаље *prompt*-ове, прима и обрађује одговоре, и враћа резултате контролерима у облику који одговара *DTO* моделима из *Core* слоја.

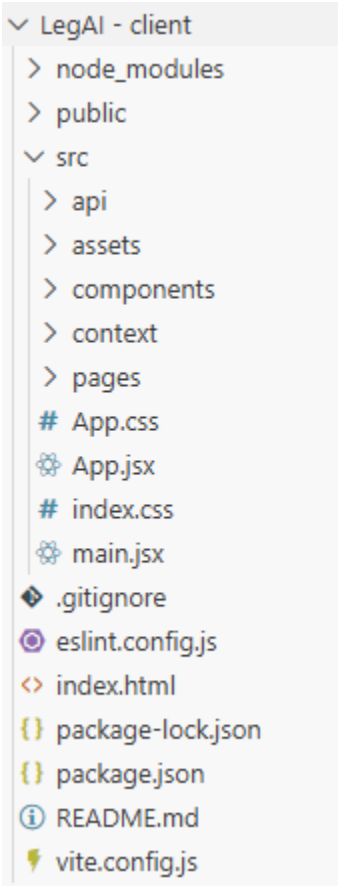
На овај начин се логика интеграције са спољним сервисом држи издвојено, а остатак система остаје независан од конкретне библиотеке или начина позивања.

• ***Security***

Фолдер *Security* обухвата класе повезане са безбедношћу, као што је механизам за хеширање лозинки. Задатак овог слоја је да обезбеди да се осетљиви подаци (попут лозинки) никада не чувају у бази у чистом тексту, већ искључиво у хешираном облику.

8.4. Имплементација *frontend*-а апликације

Кориснички интерфејс веб решења (енг. *frontend*) *LegAI* развијен је коришћењем *React* библиотеке, уз *Vite* као окружење за брз развој и оптимизовано стварање коначне верзије апликације. Комплетна логика корисничког интерфејса налази се у директоријуму *src*, који садржи све компоненте, странице, контекст за управљање стањем и модуле за комуникацију са *backend*-ом. На слици 8.4.1. је приказан садржај предње стране апликације.



Слика 8.4.1. *Приказ садржаја предње стране апликације.*

Најбитнији фолдери и фајлови предње стране апликације:

Фолдер src:

- **api:** садржи модуле који дефинишу све *HTTP* позиве према *backend API*-ју. Ови фајлови садрже функције за регистрацију и пријаву корисника, преузимање и креирање шаблона, генерисање докумената, рад са историјом и позиве *OpenAI* сервису за објашњења правних клаузула и препоруку одговарајућих шаблона.
- **assets:** садржи статичке ресурсе као што су слике, иконице и стилови који се користе у одређеним компонентама.
- **components:** представља збирку поново употребљивих *React* компоненти које граде структуру апликације. У њему се налазе елементи попут *NavBar.jsx*, који дефинише горњу навигацију, *Layout.jsx* који обједињује распоред странице и укључује заједничке делове интерфејса, *Footer.jsx* који чини подножје странице и *SidePanel.jsx* који се приказује након пријаве корисника и олакшава кретање кроз апликацију.
- **context:** налази се *AuthContext.jsx*, датотека која управља глобалним стањем аутентификације. Овде се чувају подаци о пријављеном кориснику, његов *JWT* токен и логика за одјаву, па све компоненте могу лако да приступе овим подацима без прослеђивања кроз *props (properties)* механизам.

- **pages:** садржи главне странице апликације, односно целине које представљају функционалности доступне корисницима, то су:
    - ***Home.jsx*** приказује почетну страну апликације и њене опште информације.
    - ***Login.jsx*** и ***Register.jsx*** реализују форме за пријаву и регистрацију.
    - ***Dashboard.jsx*** служи као почетна страница након пријаве, где се кориснику нуде брзи преглед и приступ главним функцијама.
    - ***AdminUser.jsx*** приказује листу свих регистрованих корисника у систему, статистику корисника и генерисаних докумената.
    - ***Templates.jsx*** приказује листу свих шаблона са могућношћу претраге и филтрирања.
    - ***TemplateCreate.jsx*** омогућава администратору да додаје и брише шаблоне.
    - ***Generate.jsx*** омогућава интерактивно попуњавање података кроз "*wizard*" процес и представља једну од најважнијих страница система.
    - ***Documents.jsx*** приказује историју свих генерисаних докумената са могућношћу поновног преузимања.
    - ***AiAssistant.jsx*** пружа интерфејс за комуникацију са вештачком интелигенцијом која објашњава правне појмове или предлаже шаблоне на основу описа ситуације.

На нивоу апликације, *App.jsx* има улогу централне компоненте која дефинише рутирање између страница и обједињује заједничке делове интерфејса, као што су *Layout* и навигација.

Датотека *main.jsx* представља улазну тачку у којој се *React* апликација иницијализује и повезује са *HTML* документом. Глобални стилови дефинисани су у *index.css* и *App.css* фајловима, док *index.html* садржи основну структуру веб странице у коју се *React* учитава.

8.5. Случајеви коришћења

Веб решење *LegAI* подржава два основна типа корисника: непријављеног корисника, који у систему функционише као посетилац, и регистрованог корисника, који може бити обичан корисник или администратор. Свака улога има јасно дефинисане могућности и ограничења, у складу са нивоом приступа и наменом у оквиру апликације.

Посетилац има најмањи обим функционалности, али довољан да стекне утисак о томе како систем функционише. Као посетилац, он може да прегледа основне информације о апликацији и њеној намени, као и да види листу доступних шаблона докумената са кратким описима. На располагању су му и страница за регистрацију налога и форма за пријаву у систем, што омогућава брз прелазак на напредније функционалности.

Регистровани корисник представља централну улогу у систему и има приступ свим кључним механизмима за креирање правних докумената. У свом профилу може да прегледа и ажурира личне податке, који се аутоматски убацују у нове документе ради бржег попуњавања. Након пријаве, корисник може да приступи листи доступних шаблона и да их филтрира у складу са категоријом или називом. Када изабере шаблон, покреће се интерактивни „*wizard*“ који на основу *placeholder* поља у шаблону аутоматски формира

формулар за попуњавање. По завршетку уноса, систем генерише документ у *DOCX* формату и чува га у историји, одакле корисник касније може поново да га преузме, промени или обрише. Поред тога, регистровани корисник има приступ *AI* асистенту који пружа једноставна објашњења правних појмова и клаузула путем *chat*-а, као и препоруке одговарајућих шаблона на основу описане ситуације. Историја свих *chat*-ова се памти и корисник се може у било ком тренутку вратити у неки од претходних *chat*-ова, и наставити конверзацију са *AI* асистентом. Након завршетка рада, корисник се може одјавити и безбедно напустити систем.

Администратор има проширене надлежности и, поред свих функција доступних регистрованом кориснику, управља и шаблонима докумената. Његова улога је да одржава квалитет и ажурност правних образаца. Он може да креира нове шаблоне путем форме у којој уноси назив, категорију, опис и прилаже правни документ са екстензијом *.docx*. Уколико је потребно, администратор може да обрише постојеће шаблоне, како би спречио кориснике да користе заостале документе. Поред рада са шаблонима, администратор има увид у листу корисника и статистику корисника и генерисаних докумената.

Сваки непријављени корисник има следеће могућности:

- Преглед основних информација о апликацији.
- Преглед листе доступних шаблона докумената.
- Регистрација новог корисничког налога уколико већ не постоји.
- Пријава на систем уносом имејл адресе и лозинке.

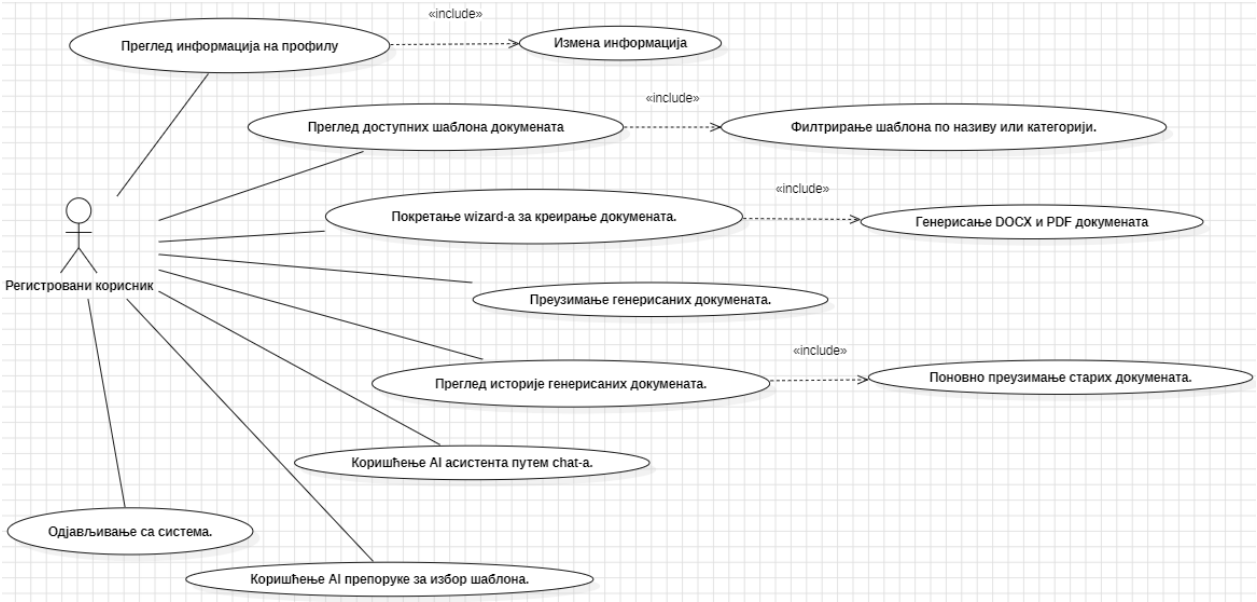
На слици 8.5.1. налази се приказ случајева коришћења за непријављеног корисника.

Слика 8.5.1. *Дијаграм случајева коришћења за Непријављеног Корисника.*

Регистровани корисник осим могућности које има непријављени корисник има следеће могућности:

- Преглед и измена профилних података.
- Преглед доступних шаблона докумената.
- Филтрирање шаблона по категорији или називу.
- Покретање wizard-а за креирање документа.
- Генерисање DOCX документа.
- Преузимање генерисаних докумената.
- Преглед историје генерисаних докумената.
- Поновно преузимање старих докумената.
- Коришћење AI помоћника за објашњење правних клаузула путем chat-а.
- Коришћење AI препоруке за избор шаблона.
- Одјављивање са система.

На слици 8.5.2. налази се приказ случајева коришћења за регистрованог корисника.



Слика 8.5.2. *Дијаграм случајева коришћења за Регистрованог Корисника.*

Администратор има све могућности као и регистровани корисник, али и додатне опције управљања:

- Креирање новог шаблона документа.
- Брисање постојећег шаблона.
- Преглед листе регистрованих корисника.
- Одјављивање са система.

На слици 6.6.1. налази се приказ случајева коришћења за администратора.

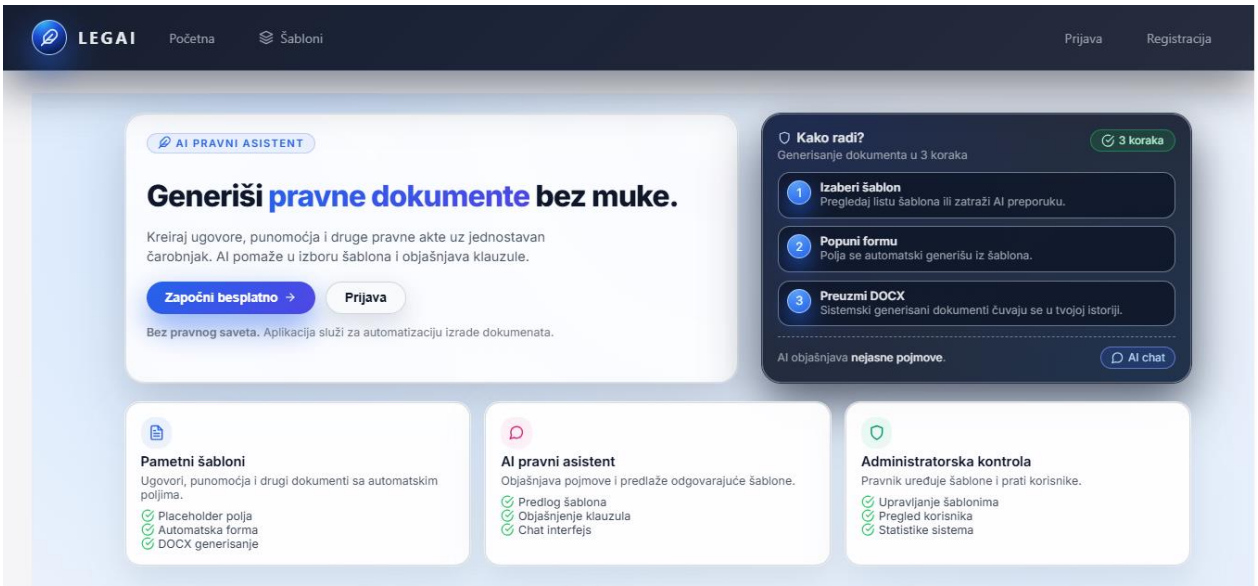
Слика 8.5.3. *Дијаграм случајева коришћења за Администратора.*

9. Приказ рада апликације

У овом поглављу приказан је рад апликације и кориснички интерфејс.

9.1. Почетна страница

Када корисник покрене апликацију, прва страница која се приказује је почетна страница веб решења Leg*AI*. На слици 9.1.1. приказана је почетна страница.

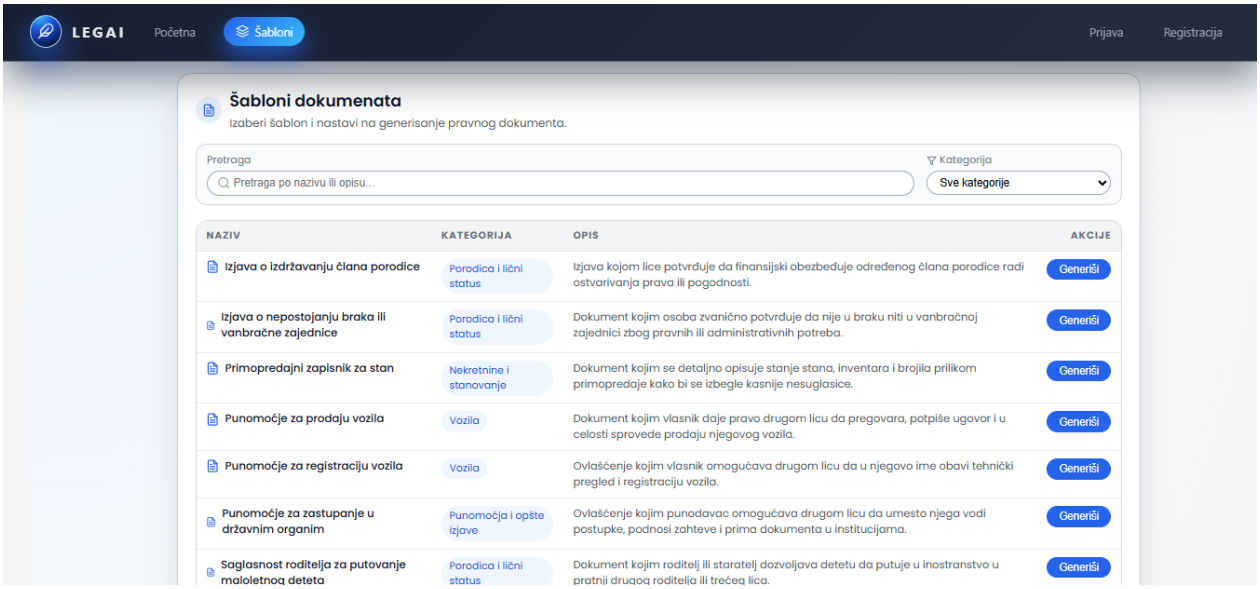


Слика 9.1.1. *Приказ почетне странице.*

Корисник има могућност да се региструје, уколико нема креиран налог или да помоћу навигационог бара прегледа листу доступних шаблона. Уколико корисник већ има креиран налог, онда се може пријавити на свој налог.

9.2. Страница „*Šabloni*“

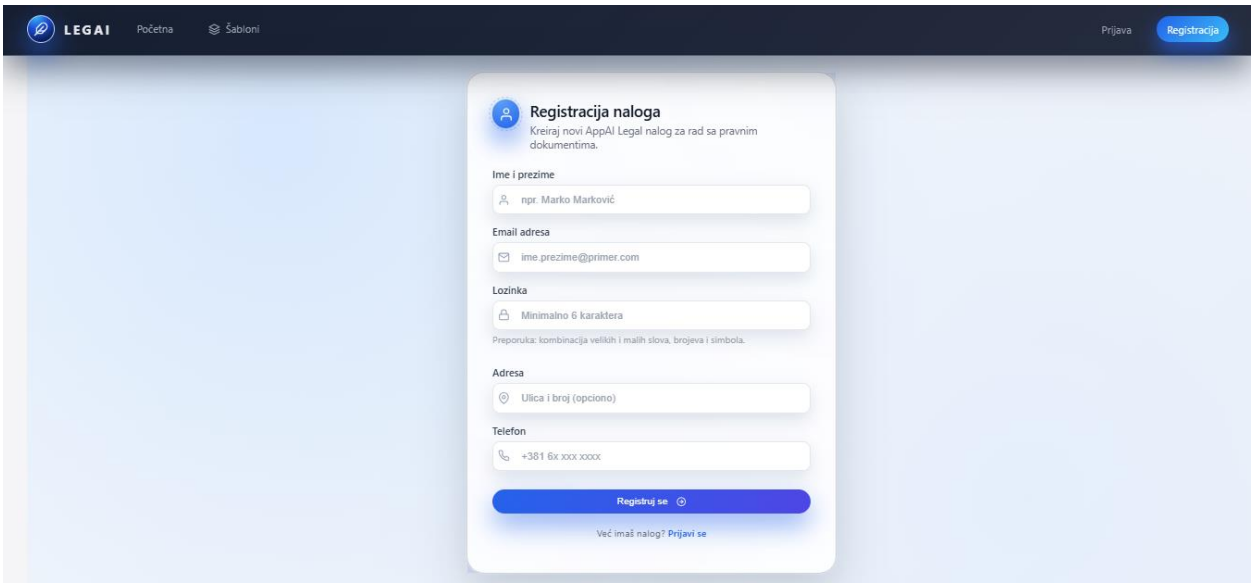
Кликом на „*Šabloni*“ у навигационом бару приказује се страница са свим доступним шаблонима за коришћење у систему. Кликом на дугме „*Generiši*“, корисник се преусмерава на страницу „*Prijava*“. Уколико корисник није регистрован и пријављен нема могућност генерисања докумената, нити видљивост и коришћење *AI* асистента. На слици 9.2.1. приказана је страница са шаблонима.



Слика 9.2.1. *Приказ странице са шаблонима.*

9.3. Страница за регистрацију корисника

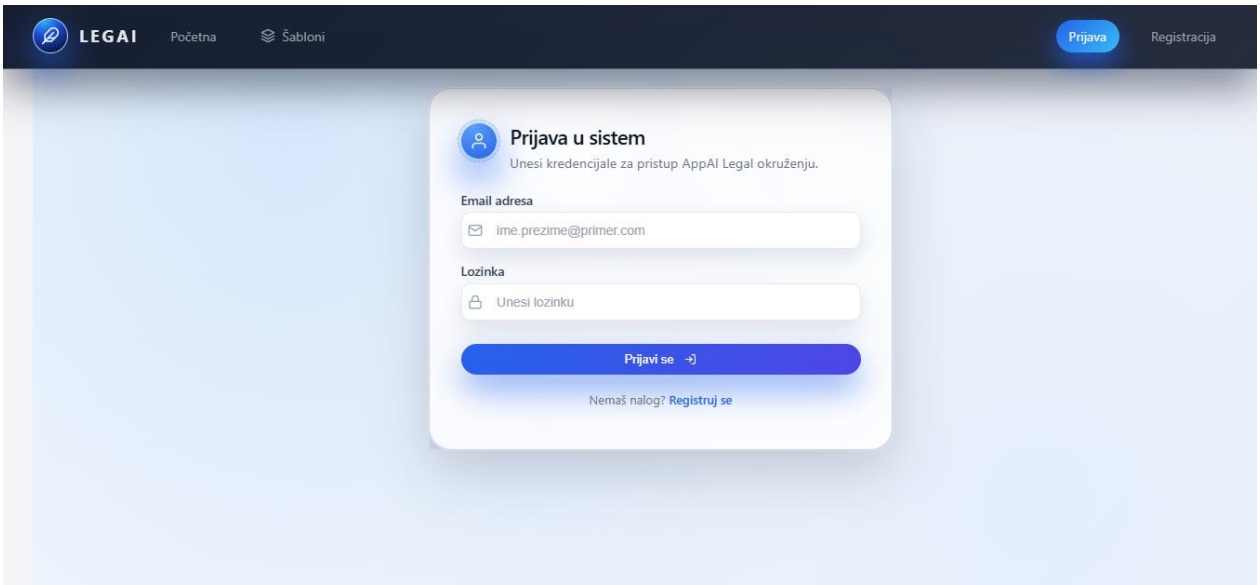
Уколико корисник нема свој налог, може се регистровати кликом на дугме „*Započni besplatno*“ на почетној страници или избором опције „*Registracija*“ у навигационом бару. Корисник треба да попуни основне податке како би успешно завршио процес регистрације. На слици 9.3.1 приказана је страница са формом за регистрацију.



Слика 9.3.1. *Приказ странице са формом за регистрацију.*

9.4 Страница за пријаву корисника

Након што се корисник успешно региструје, он може приступити свом налогу кликом на опцију „*Prijava*“ у навигационом бару, чиме се отвара форма за пријављивање. Након уношења својих корисничких података, корисник добија приступ свом налогу. На Слици 9.4.1. приказана је страница са формом за пријаву.



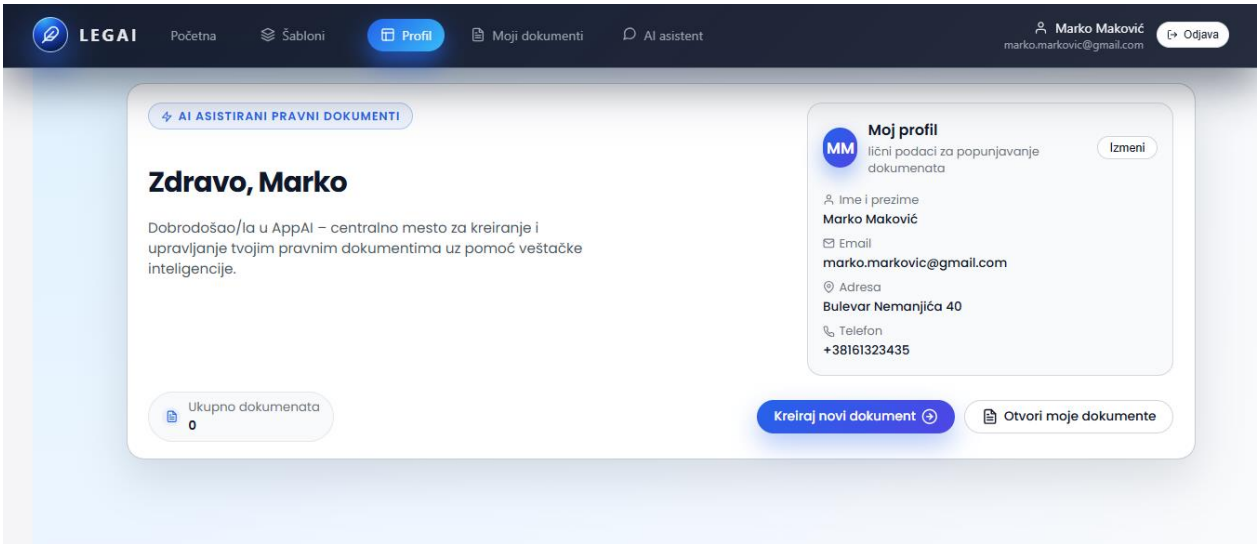
Слика 9.4.1. *Приказ странице са формом за пријављивање.*

9.5. Кориснички профили

Након успешне пријаве, корисник се аутоматски преусмерава на свој профил.

9.5.1. Профил корисника

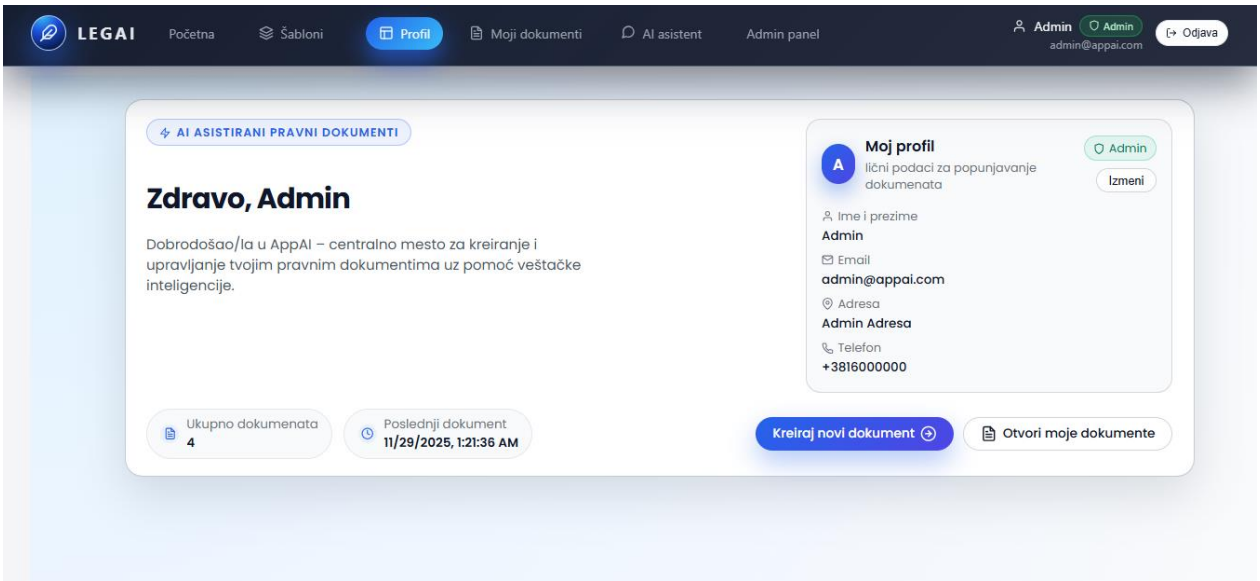
Картица „*Moj profil*“ приказује личне податке корисника. На дну леве стране профила налази се укупан број генерисаних докумената корисника. На дну десне стране налази се дугме „*Kreiraj novi dokument*“ које води корисника на страницу „*Šabloni*“, и дугме „*Otvori moje dokumente*“ којим се отвара страница „*Moji dokumenti*“ на којој се налази историја генерисаних докумената корисника. На слици 9.5.1.1. је приказан профил регистрованог корисника, чија је улога обичан корисник.



Слика 9.5.1.1. *Приказ профила корисника.*

9.5.2. Профил администратора

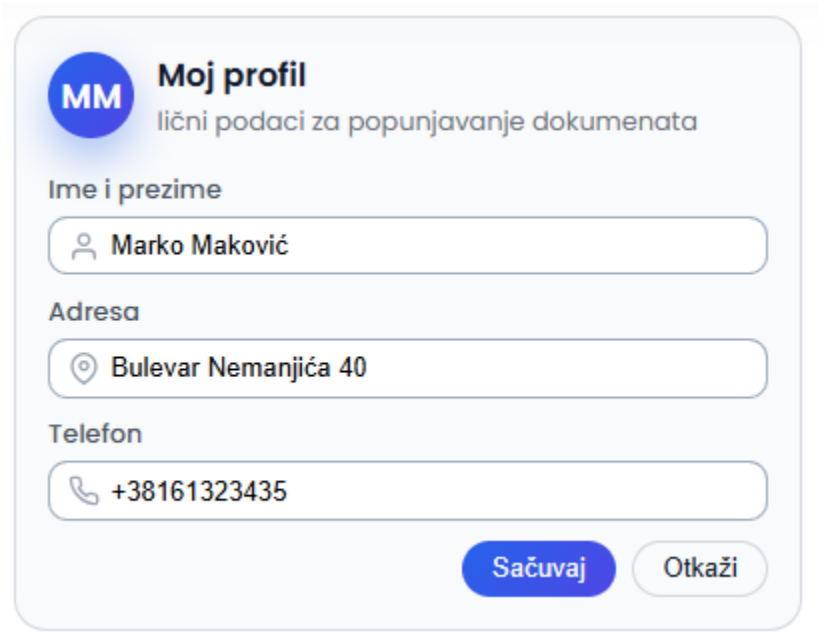
На профилу администратора се, поред могућности и информације које се приказују на профилу обичног корисника, приказује и датум када је последњи шаблон креиран. На слици 9.5.2.1. је приказан профил регистрованог корисника, чија је улога администратор. У систему постоји искључиво један администратор, који представља „правно“ лице.



Слика 9.5.2.1. *Приказ профила администратора*

9.5.3. Измена личних података

Корисници (укључујући и администратора) имају могућност измене личних података кликом на дугме „*Izmeni*“, које се налази са десне стране у картици „*Moj profil*“, и отвара се форма за измену личних података. Кликом на дугме „*Sačuvaj*“ биће сачуване унете измене. На слици 9.5.3.1. је приказана форма за измену личних података.

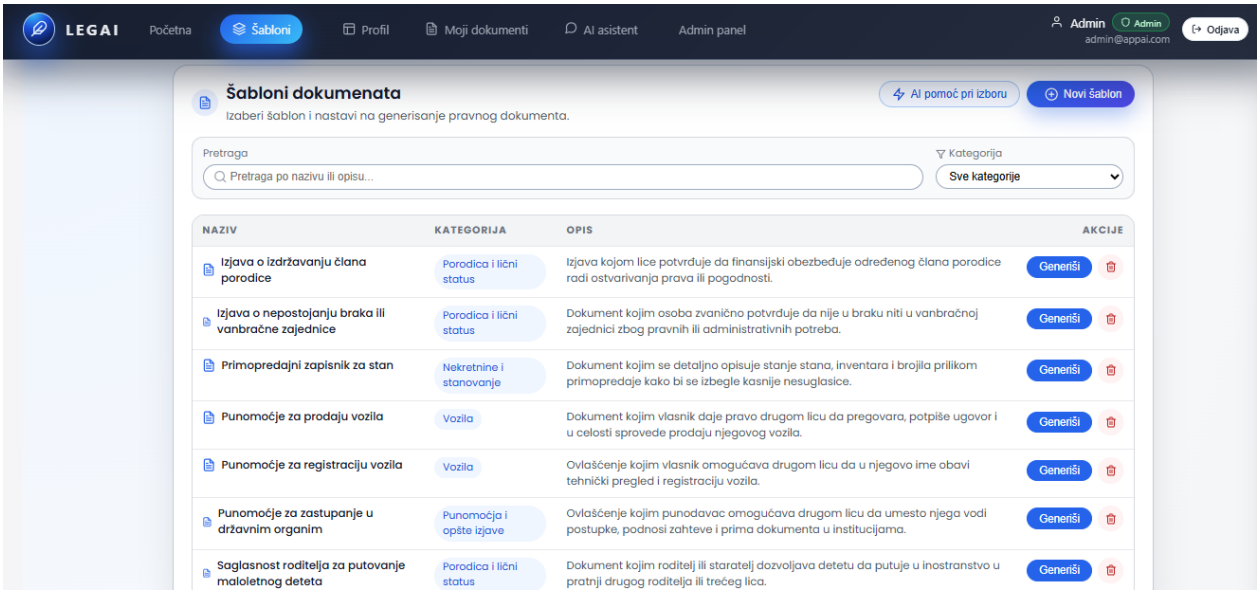


Слика 9.5.3.1. *Приказ форме за измену личних података..*

9.6 Шаблони правних докумената

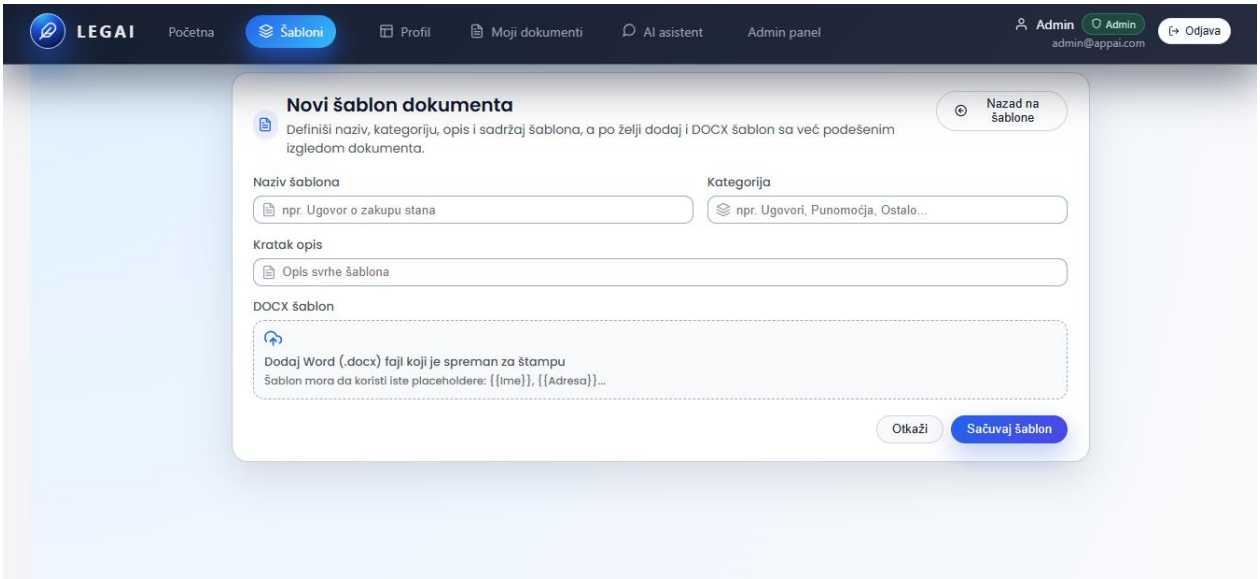
9.6.1. Генерисање шаблона правних докумената

Администратор, који представља „правно“ лице је једини корисник у систему који може генерисати шаблоне правних докумената. Када је администратор пријављен, он једини има могућност видљивости дугмета „*Novi šablon*“ на страници „*Šabloni*“. На слици 9.6.1.1. је приказана страница шаблона када је администратор пријављен.



Слика 9.6.1.1. *Приказ странице шаблона када је администратор пријављен.*

Кликом на дугме „*Novi šablon*“ отвара се форма за креирање новог шаблона правног документа. На слици 9.6.1.2. је приказ форме за креирање новог шаблона.



Слика 9.6.1.2. *Приказ форме за креирање новог шаблона.*

Администратор попуњава потребне информације, и поставља фајл правног документа, екстензије *.docx* са диска у секцији „*DOCX šablon*“. Тај фајл представља шаблон правног документа. Важно је да у том документу администратор поља, у којима обичан корисник уноси податке, декларише као *placeholder*-и (нпр. {{*Ime*}}, {{*Adresa*}}, итд.). Кликом на дугме „*Sačuvaj šablon*“ шаблон ће бити сачуван у бази и биће одмах видљив на страници „*Šabloni*“.

На слици 9.6.1.3. је пример структуре правног документа који администратор поставља при креирању шаблона.

UGOVOR O POKLONU NEPOKRETNOSTI

Koji zaključuju dana {{DatumZakljucivanjaUgovora}} godine ugovorne strane :
Poklonodavac : {{ImePrezimePoklonodavca}} iz {{PrebilavistePoklonodavca}}, JMBG {{JMBGPoklonodavca}}.
Poklonoprimac : {{ImePrezimePoklonoprimac}} iz {{PrebivalistePoklonoprimaca}}, JMBG {{JMBGPoklonoprimca}}.

Član 1.

Poklonodavac je zemljišnoknjižni vlasnik nepokretnosti i na osnovu onog Ugovora poklanja Poklonoprimcu, vlasništvo nad {{ImeParcele}} u površini od {{PovrsinaPacele}} ara, na katastarskoj parceli br. {{BrojParcele}} koja je upisana u listu nepokretnosti br {{BrListNepokretnosti}}.

Član 2.

Poklonodavac poklanja Poklonoprimcu nepokretnost opisanu u članu 1. ovog Ugovora bez ikakve novčane ili druge naknade, u stanju u kome se ova nepokretnost nalazi na dan poklona i bez ikakvih tereta.
Poklonodavac, potpisom ovog Ugovora, izričito ijavljuje da se nakon njegove smrti, predmet poklona utvrđen članom 1. ovog Ugovora neće uračunavati u nasledni deo.
Poklonoprimac prima poklonjenu nepokretnost sa zahvalnošću.
Porez na promet apsolutnih prava i sve takse u vezi zaključivanja ovog ugovora padaju na teret Poklonoprimca.

Član 3.

Poklonodavac na stanu opisanom u članu 1 ovog Ugovora zadržava za sebe pravo doživotnog uživanja i stanovanja (plodouživanja) sa pravom upisa ovog plodouživanja u katastar nepokretnosti kao teret na poklonjenoj nepokretnosti.
Poklonodavac nema pravo da bez izričite pismene saglasnosti Poklonoprimca primi treće lice na stanovanje ili duži boravak u stanu koji je predmet ovog poklona.
Poklonoprimac se obavezuje da Poklonodavca nakon njegove smrti o svom trošku sahrani prema običajima i mesnim prilikama.

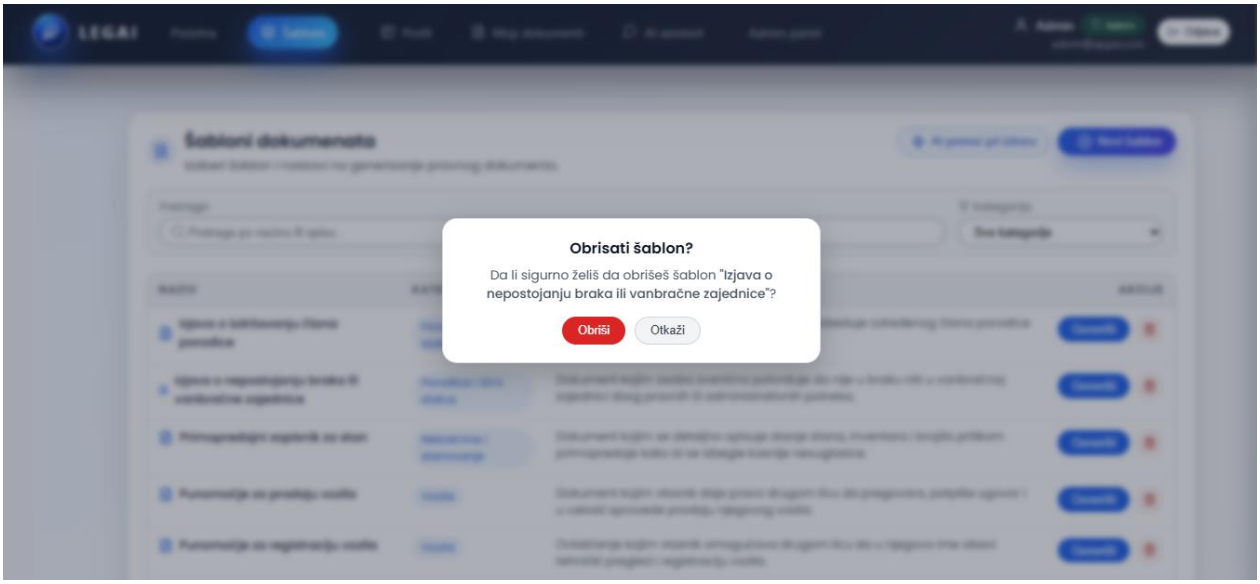
Ugovor je sačinjen u četiri (4) istovetnih primeraka od kojih svaka ugovorna strana zadržava po dva (2) primerka a završen je zaključno sa ovom klauzulom.

Poklonodavac | Poklonoprimac

Слика 9.6.1.3. *Пример правног документа.*

9.6.2. Брисање постојећих шаблона

Поред генерисања шаблона, администратор има могућност и брисања шаблона уколико су шаблони застарели или невалидни. Кликом на црвену кантицу приказује се прозор брисања шаблона. Затим, кликом на дугме „*Obriši*“ на прозору, шаблон ће бити трајно и неповратно обрисан из система. На слици 9.6.2.1. је приказ прозора брисања шаблона.



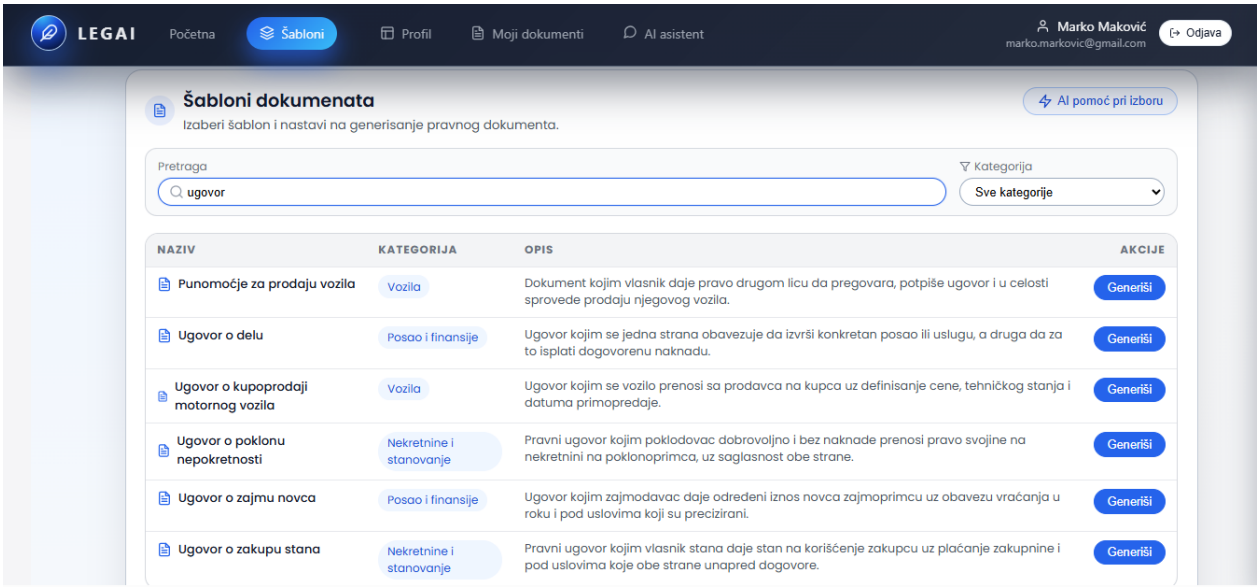
Слика 9.6.2.1. *Приказ прозора брисања шаблона.*

9.6.3. Претрага шаблона

Корисници имају могућност претраживања шаблона по називу или опису и по категоријама.

9.6.3.1 Претрага шаблона по називу или опису

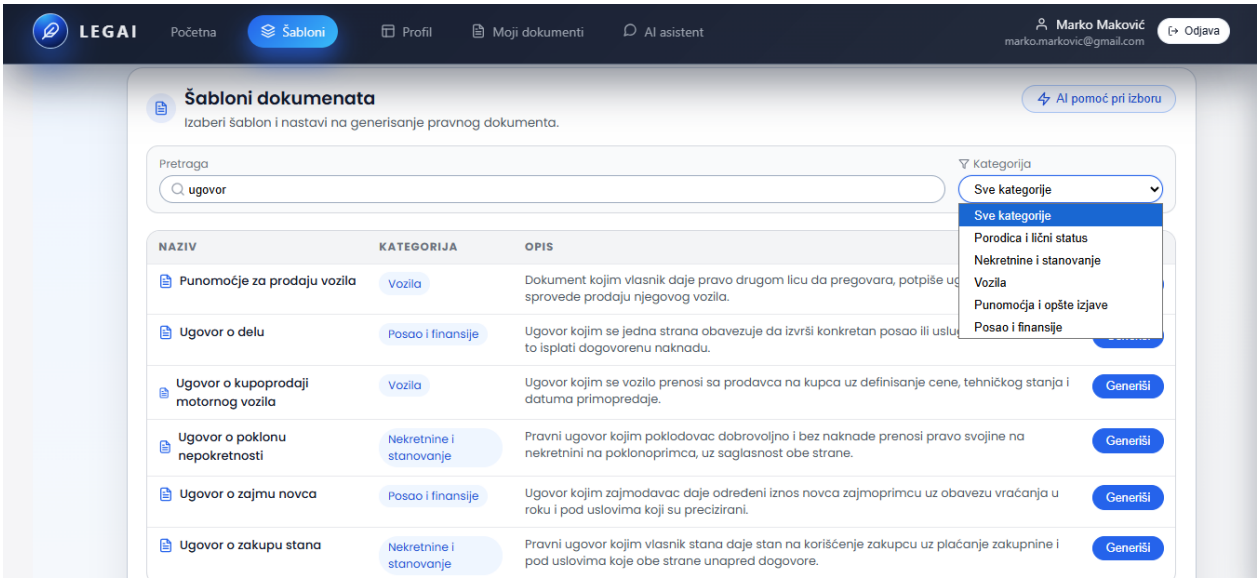
Корисник може претраживати шаблоне по називу или њиховом опису уколико није сигуран у назив тог шаблона. Систем ће приказати сва документа која одговарају том називу или опису. На слици 9.6.3.1.1. је приказана претрага шаблона по називу или опису уношењем речи „*ugovor*“ у поље за претрагу.



Слика 9.6.3.1.1. *Приказ претраге шаблона по називу или опису*

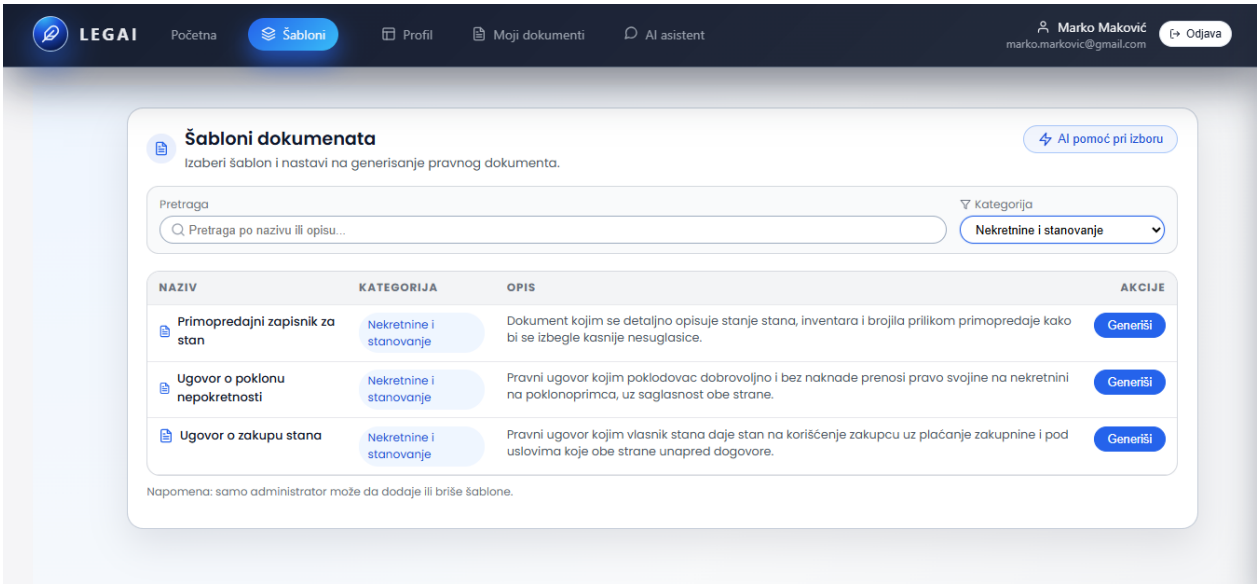
9.6.3.2. Претрага шаблона по категорији

Поред могућности претраге шаблона по називу или опису, корисник има могућност претраге шаблона по њиховој категорији. Сви шаблони система подељени су у категорије, које администратор дефинише при генерисању шаблона. На слици 9.6.3.2.1. је приказ тренутно доступних категорија у бази.



Слика 9.6.3.2.1. *Приказ доступних категорија у бази.*

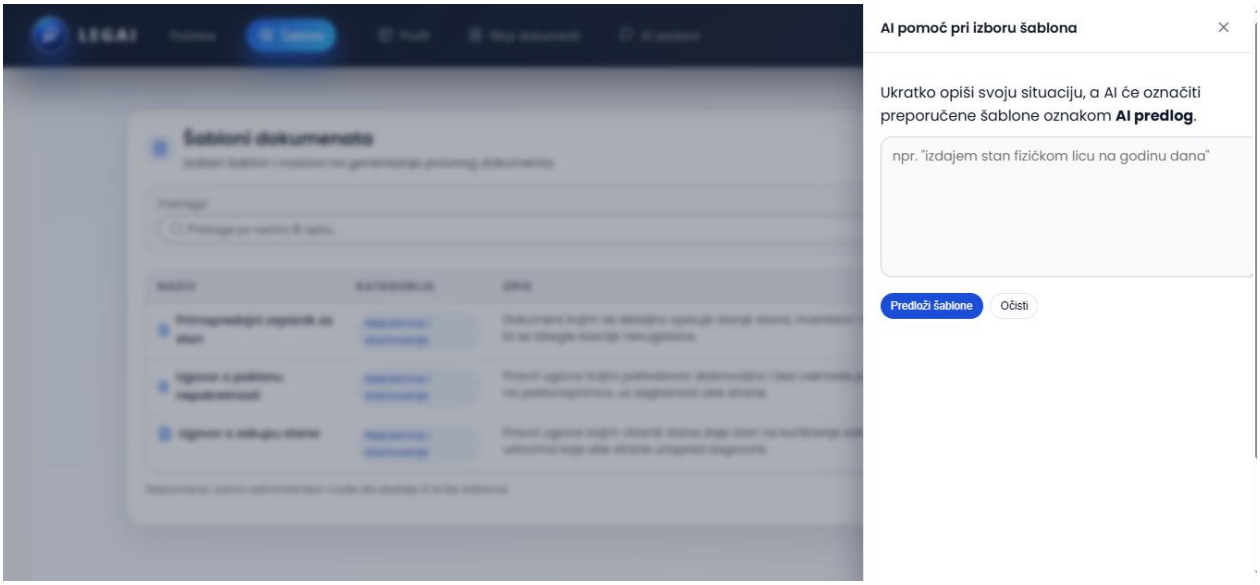
На слици 9.6.3.2.2. је приказ резултата претраге шаблона који одговарају категорији „*Nekretnine i stanovanje*“.



Слика 9.6.3.2.2. *Приказ шаблона који одговарају категорији „Nekretnine i stanovanje“.*

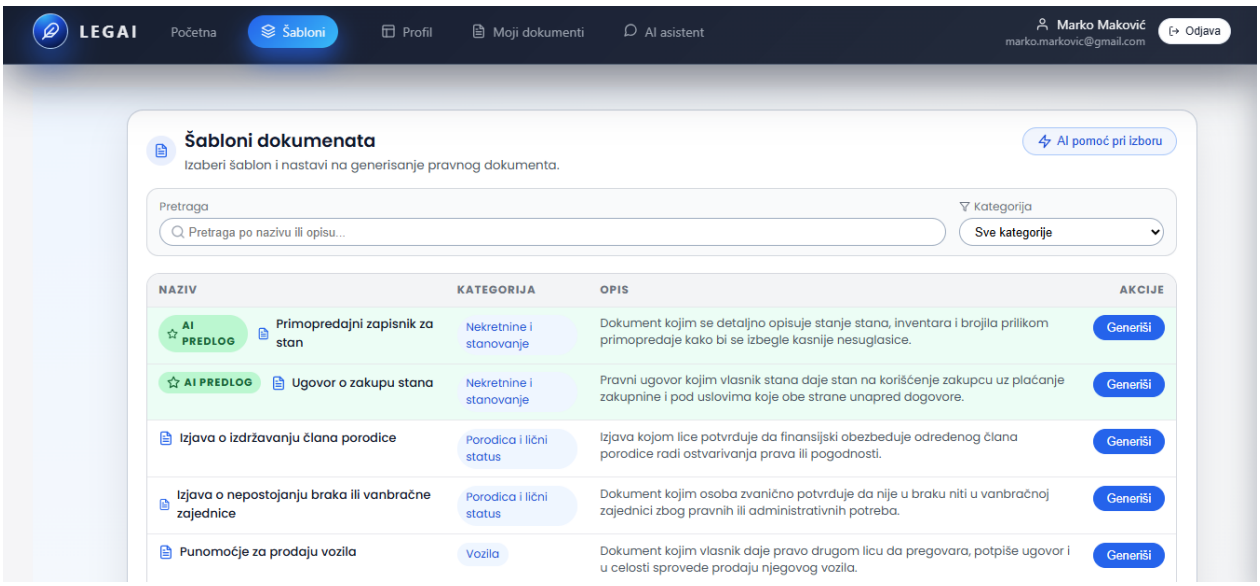
9.6.4. AI препорука шаблона

Сваки корисник, који је пријављен, има могућност да добије препоруку од вештачке интелигенције (*AI*-а) уколико није сигуран који шаблон одговара његовим потребама. Кликом на дугме „*AI preporuka šablona*“, на станици „*Šabloni*“, отвара се бочни панел са десне стране, у коме корисник описује своју ситуацију за коју му је потребан одговарајући шаблон. На слици 9.6.4.1. је приказ бочног панела *AI* препоруке.



Слика 9.6.4.1. *Приказ бочног панела AI препоруке.*

Након што корисник опише своју ситуацију, и кликне на дугме „*Prikaži šablone*“, на страници „*Šabloni*“ се приказују сви фајлови које *AI* сматра да одговарају описаној ситуацији. Фајлови које је *AI* препоручио означени су као „*AI PREDLOG*“, зеленом бојом и приказују се на врху листе свих шаблона. На слици 9.6.4.2. је приказ *AI* препоруке шаблона за пример ситуације „*издајем стан физичком лицу, препоручи ми шаблоне*“.



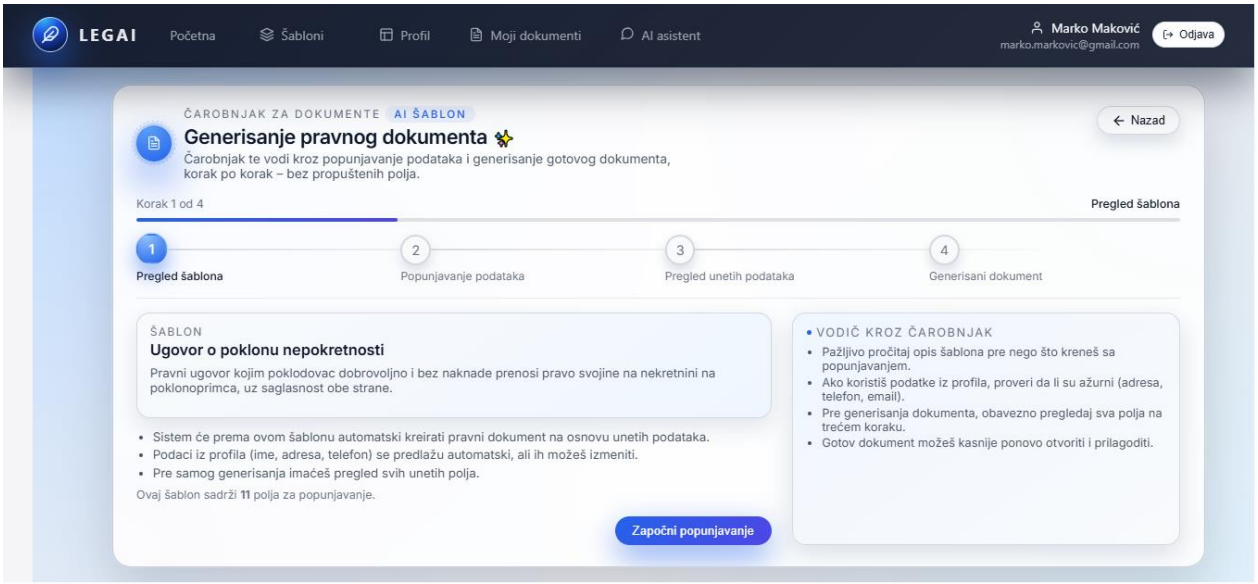
Слика 9.6.4.2. *приказ AI препоруке шаблона.*

9.6.5 Чаробњак за аутоматско генерисање докумената

Корисници имају могућност аутоматског креирања правних докумената помоћу дефинисаних шаблона. На страници „*Šabloni*“ кликом на дугме „*Generiši*“ отвара се интерактивни чаробњак за генерисање правних докумената. Чаробњак води корисника кроз процес попуњавања у четири корака.

9.6.5.1. Први корак интерактивног чаробњака

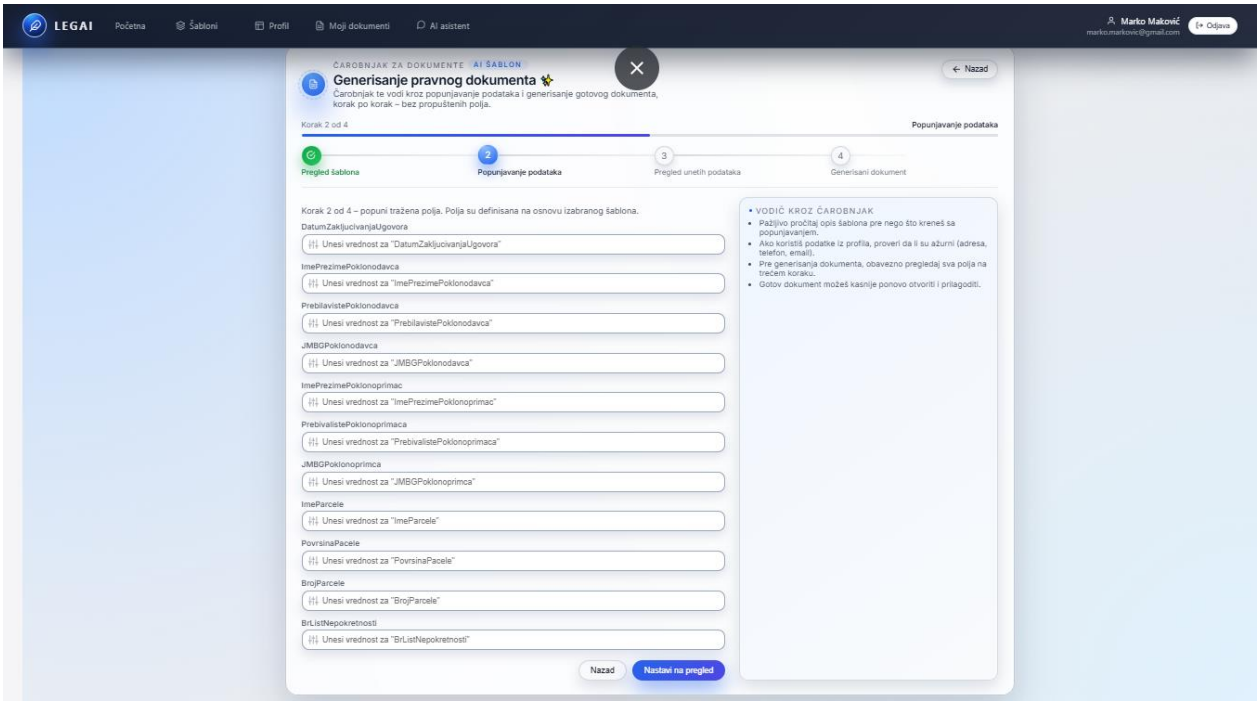
У првом кораку приказује се водич кроз чаробњак, назив и опис изабраног шаблона којим ће бити генерисан правних документ. На слици 9.6.5.1.1. је приказ првог корака интерактивног чаробњака за изабрани шаблон „*Ugovor o poklonu nepokretnosti*“.



Слика 9.6.5.1.1. *Приказује први корак интерактивног чаробњака.*

9.6.5.2. Други корак интерактивног чаробњака

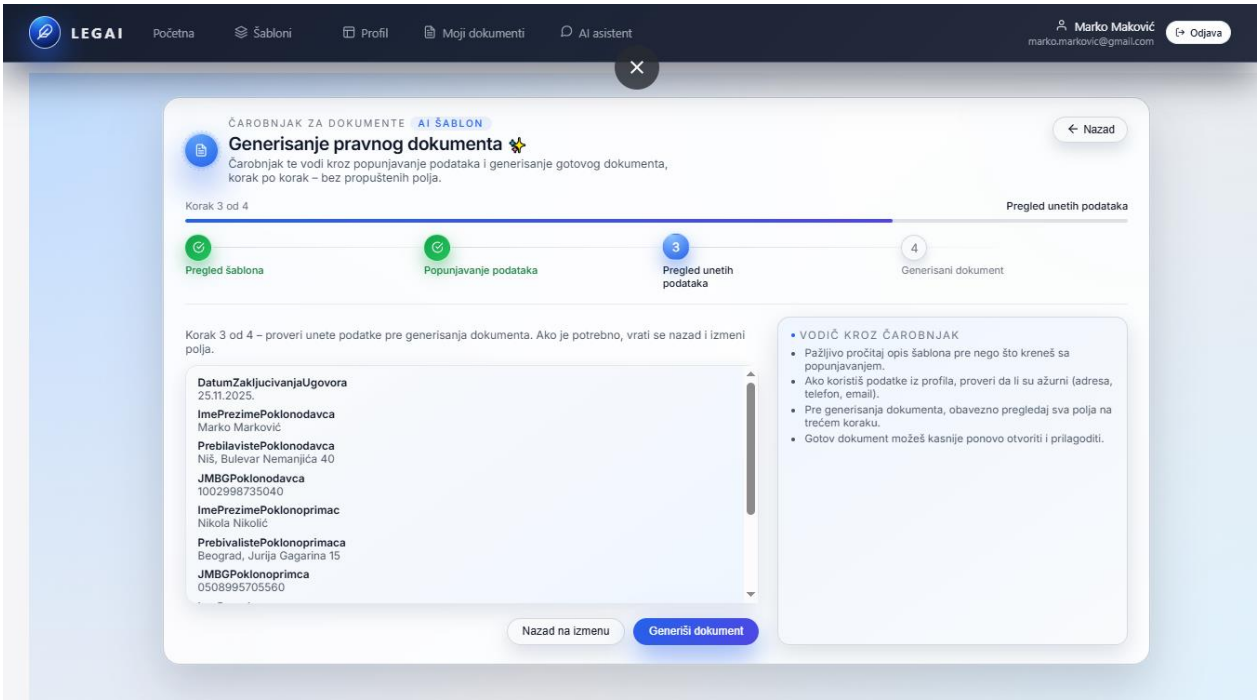
Кликом на дугме „*Započni popunjavanje*“ у првом кораку, корисник се преусмерава на други корак интерактивног чаробњака. У овом кораку отварају се поља која је потребно попунити за правни документ. Ова поља нису исто декларисана у систему за сваки шаблон, већ се мењају у зависности од изабраног шаблона. На слици 9.6.5.2.1. је приказ другог корака интерактивног чаробњака за шаблон „*Ugovor o poklonu nepokretnosti*“.



Слика 9.6.5.2.1. *Приказ другог корака интерактивног чаробњака.*

9.6.5.3. Трећи корак интерактивног чаробњака

Након што корисник попуни сва потребна поља у другом кораку, кликом на дугме „*Nastavi na pregled*“ прелази на трећи корак интерактивног чаробњака. У овом кораку захтева се од корисника да провери још једном податке које је унео, пре него да генерише документ. Уколико корисник није унео сва поља исправно, има могућност да се врати на корак два, кликом на дугме „*Nazad na izmenu*“. На слици 9.6.5.3.1. је приказ трећег корака интерактивног чаробњака за шаблон „*Ugovor o poklonu nepokretnosti*“.

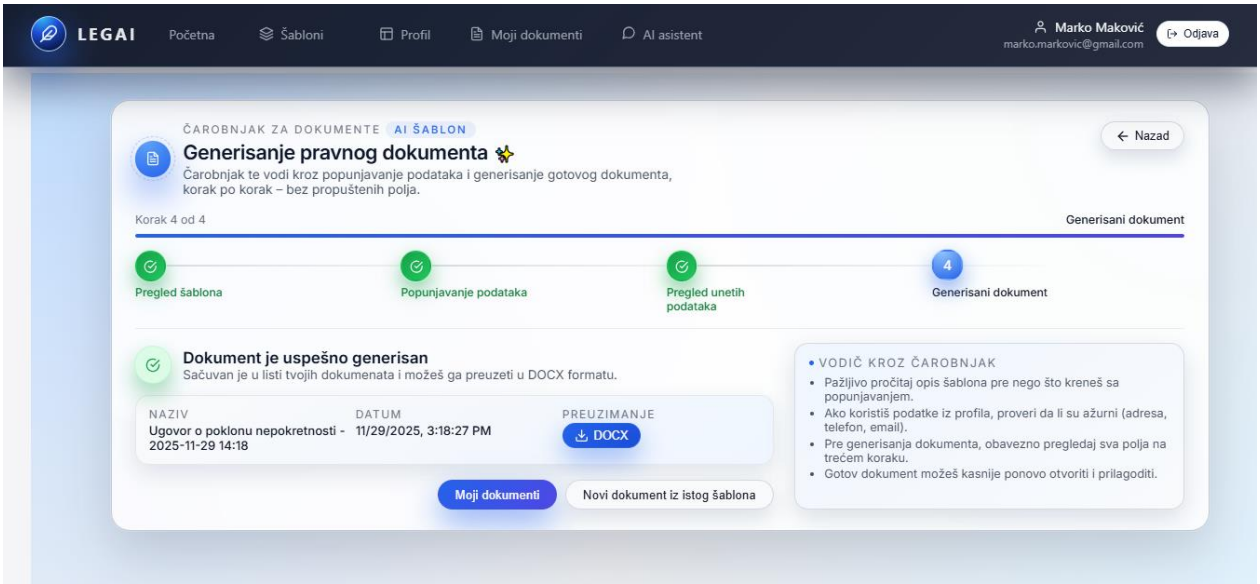


Слика 9.6.5.3.1. *Приказ трећег корака интерактивног чаробњака.*

9.6.5.4. Четврти корак интерактивног чаробњака

Након што је корисник проверио сва поља у трећем кораку, кликом на дугме „*Generiši dokument*“ корисник се усмерава на четврти корак и систем аутоматски генерише документ спреман за преузимање. У овом кораку приказује се порука о успешном генерисању документа, назив документа, датум генерисања и дугме за преузимање *.docx* верзије документа. Кликом на дугме „*DOCX*“ правни документ се преузима у *downloads* фајлу на диску. Правни документ има идентичну структуру и стил какву је администратор поставио приликом креирања ово шаблона.

Такође, у овом кораку се налази дугме „*Moji dokumenti*“ којим се корисник преусмерава на страницу „*Moji dokumenti*“ и дугме „*Novi dokument iz istog šablona*“, којим корисник може поново да се врати на први корак чаробњака и генерише нови документ за исти шаблон. На слици 9.6.5.4.1. је приказ четвртог корака интерактивног чаробњака за шаблон „*Ugovor o poklonu nepokretnosti*“.



Слика 9.6.5.4.1. *Приказ четвртог корака интерактивног чаробњака.*

На слици 9.6.5.4.2. је приказ структуре генерисаног правног документа након преузимања у четвртом кораку интерактивног чаробњака.

UGOVOR O POKLONU NEPOKRETNOSTI

Koji zaključuju dana 25.11.2025. godine ugovorne strane :
Poklonodavac : Marko Marković iz Niš, Bulevar Nemanjića 40, JMBG 1002998735040.
Poklonoprimac : Nikola Nikolić iz Beograd, Jurija Gagarina 15, JMBG 0508995705560.

Član 1.

Poklonodavac je zemljišnoknjižni vlasnik nepokretnosti i na osnovu onog Ugovora poklanja Poklonoprimcu, vlasništvo nad Beli Potok u površini od 10 ara ara, na katastarskoj parceli br. 56437 koja je upisana u listu nepokretnosti br 24345/24233.

Član 2.

Poklonodavac poklanja Poklonoprimcu nepokretnost opisanu u članu 1. ovog Ugovora bez ikakve novčane ili druge naknade, u stanju u kome se ova nepokretnost nalazi na dan poklona i bez ikakvih tereta.
Poklonodavac, potpisom ovog Ugovora, izričito ijavljuje da se nakon njegove smrti, predmet poklona utvrđen članom 1. ovog Ugovora neće uračunavati u nasledni deo.
Poklonoprimac prima poklonjenu nepokretnost sa zahvalnošću.
Porez na promet apsolutnih prava i sve takse u vezi zaključivanja ovog ugovora padaju na teret Poklonoprimca.

Član 3.

Poklonodavac na stanu opisanom u članu 1 ovog Ugovora zadržava za sebe pravo doživotnog uživanja i stanovanja (plodouživanja) sa pravom upisa ovog plodouživanja u katastar nepokretnosti kao teret na poklonjenoj nepokretnosti.
Poklonodavac nema pravo da bez izričite pismene saglasnosti Poklonoprimca primi treće lice na stanovanje ili duži boravak u stanu koji je predmet ovog poklona.
Poklonoprimac se obavezuje da Poklonodavca nakon njegove smrti o svom trošku sahrani prema običajima i mesnim prilikama.

Ugovor je sačinjen u četiri (4) istovetnih primeraka od kojih svaka ugovorna strana zadržava po dva (2) primerka a završen je zaključno sa ovom klauzulom.

Poklonodavac | Poklonoprimac

Слика 9.6.5.4.2. *Приказ структуре генерисаног правног документа.*

9.7 Документи корисника

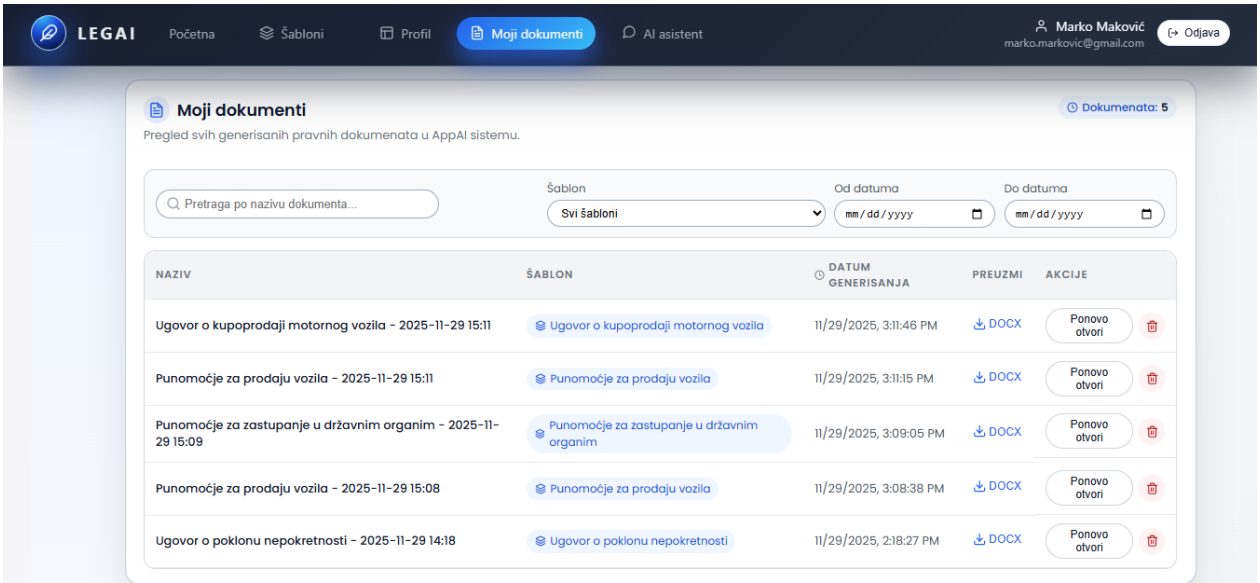
Кликом у навигационом бару на „*Moji dokumenti*“ отвара се страница која приказује историју свих докумената које је корисник генерисао.

9.7.1. Историја корисникових докумената

Корисник има могућност да види све документе које је икада генерисао. У историји докумената приказује се назив документа, шаблон и датум генерисања. Корисник може поново преузети генерисани документ, уколико му је то потребно, и сачувати га у произвољном фолдеру на свом диску, кликом на дугме „*DOCX*“ испод колоне „*PREUZMI*“.

Такође, корисник има могућност да поново отвори генерисати документ у чаробњаку, кликом на дугме „*Ponovo otvori*“, и измени жељена поља. Након сачуваних измена, у систему ће се памтити искључиво последња верзија тог документа. Неће бити креирана нова верзија у систему, већ ће само бити ажурирани подаци које је корисник променио у том документу.

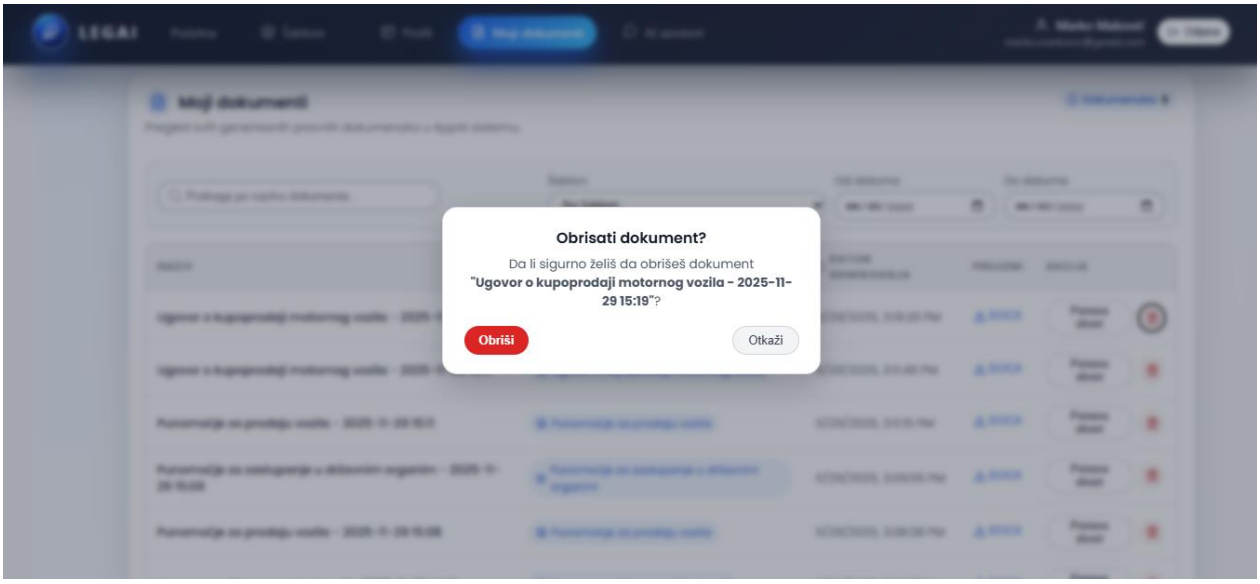
На слици 9.7.1.1. приказана је историја свих правних докумената које је корисник генерисао.



Слика 9.7.1.1. *Приказ историје правних докумената корисника.*

9.7.2. Брисање генерисаних докумената

Кориснику је омогућено брисање генерисаних докумената из историје докумената „*Moji dokumenti*“, уколико корисник сматра да му одређени фајл више није потребан. Ова акција се обавља кликом на црвену кантицу, којом се отвара прозор за брисање. Кликом на дугме „*Obriši*“ на прозору биће трајно и неповратно обрисан генерисани правни документ корисника. На слици 9.7.2.1. је приказ прозора за брисање генерисаног правног документа корисника.



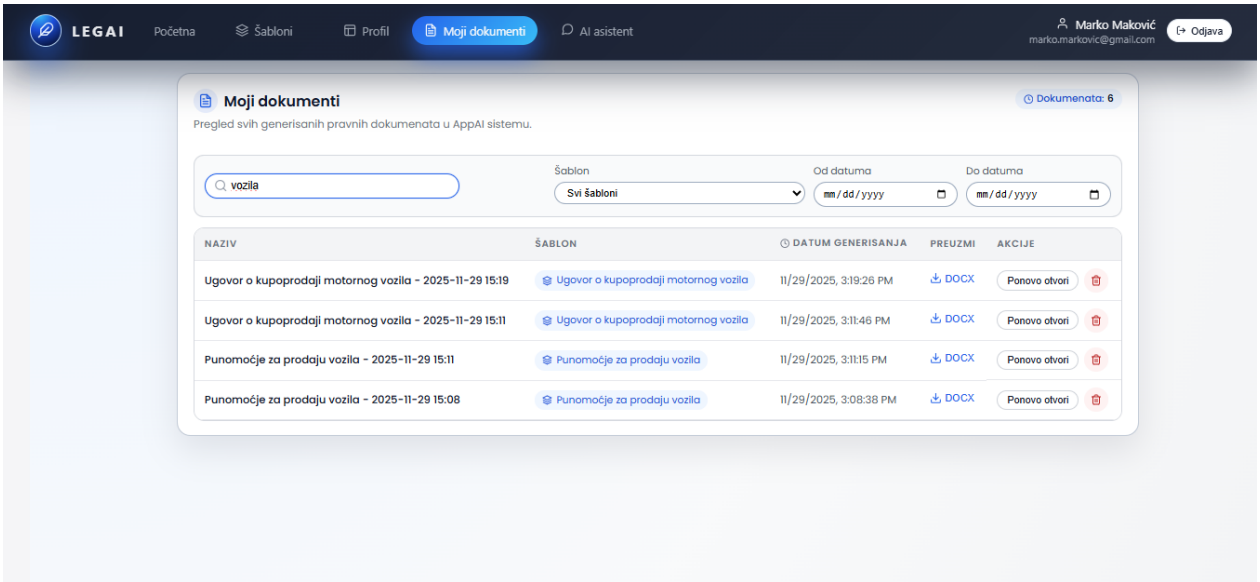
Слика 9.7.2.1. *Приказ прозора за брисање правног документа.*

9.7.3 Претрага докумената корисника

Ради бржег и лакшег коришћења ове апликације, корисницима је омогућено претраживање својих докумената. Претрагу је могуће вршити по називу, шаблону или датуму. Може се вршити претрага по једној од ове три опције, а могуће је и комбиновати све три опције претраге и пронаћи тачно одређени документ.

9.7.3.1 Претрага по називу документа

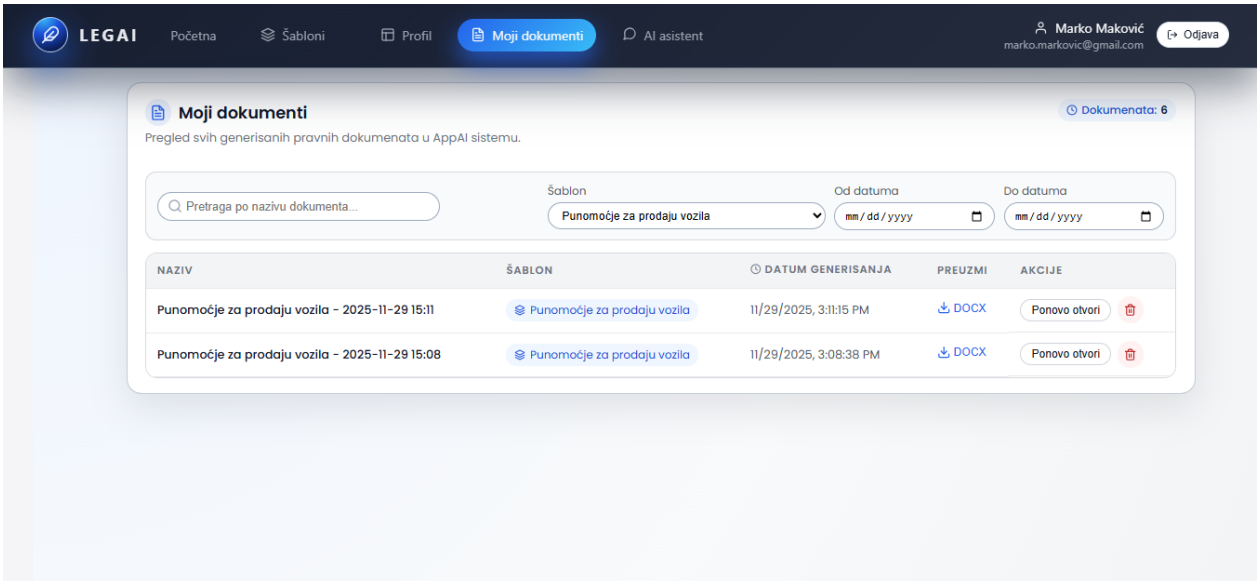
Корисник може претражити жељени документ тако што ће унети назив жељеног документа у поље за претрагу. На слици 9.7.3.1.1 је приказ резултата претраге докумената за пример назива „*vozilo*“.



Слика 9.7.3.1.1. *Приказ резултата претраге докумената по називу.*

9.7.3.2. Претрага по шаблону

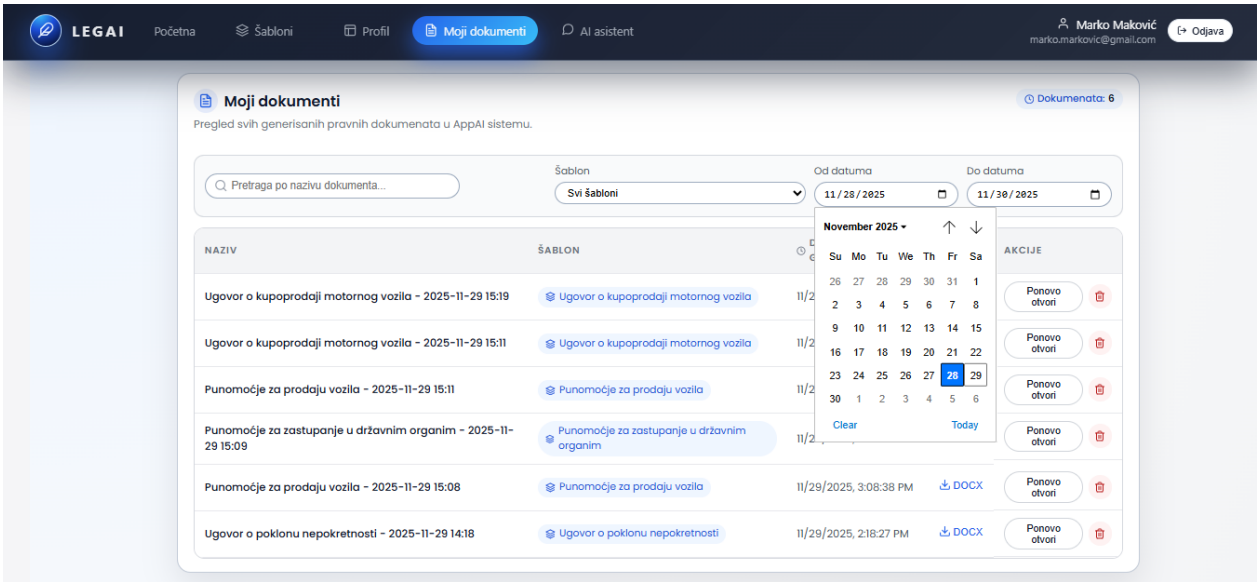
Корисник може претраживати документе помоћу назива шаблона. На слици 9.7.3.2.1. је приказ тренутно генерисаних шаблона докумената корисника у систему.



Слика 9.7.3.2.1. *Приказ резултата претраге докумената по шаблону.*

9.7.3.3. Претрага по датуму

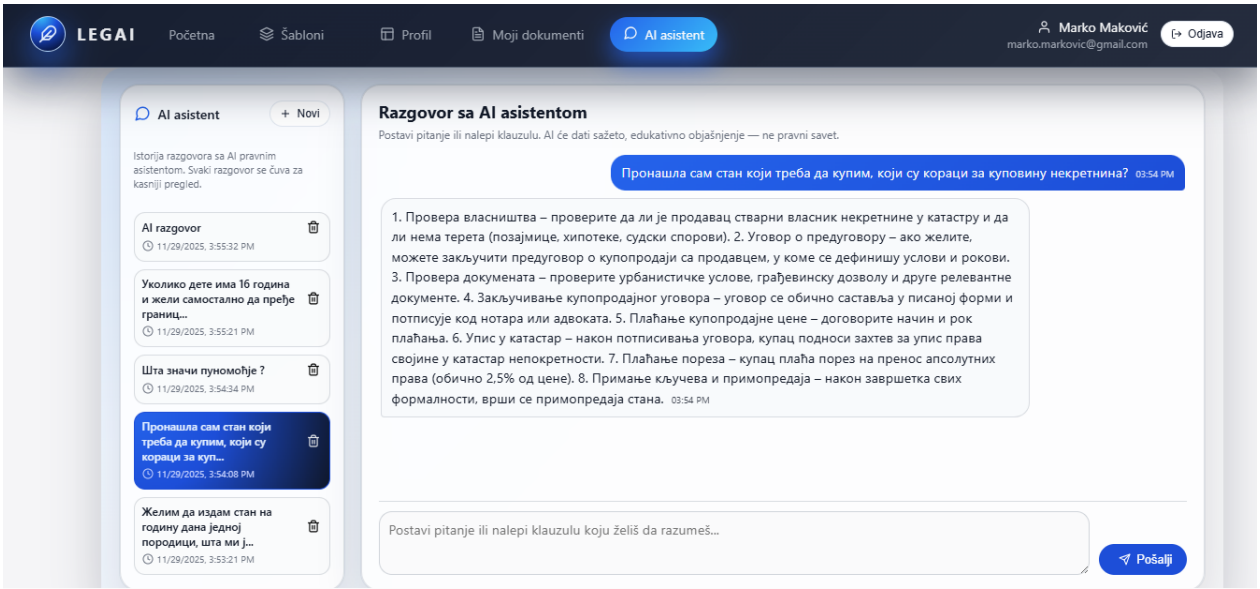
Уколико корисник жели да види све генерисане документе у одређеном период, то му је омогућено претрагом по датуму. На слици 9.7.3.3.1. је приказ резултата претраге за документе који су генерисани од 28.11.2025. до 30.11.2025.



Слика 9.7.3.3.1. *Приказ резултата претраге докумената по датуму.*

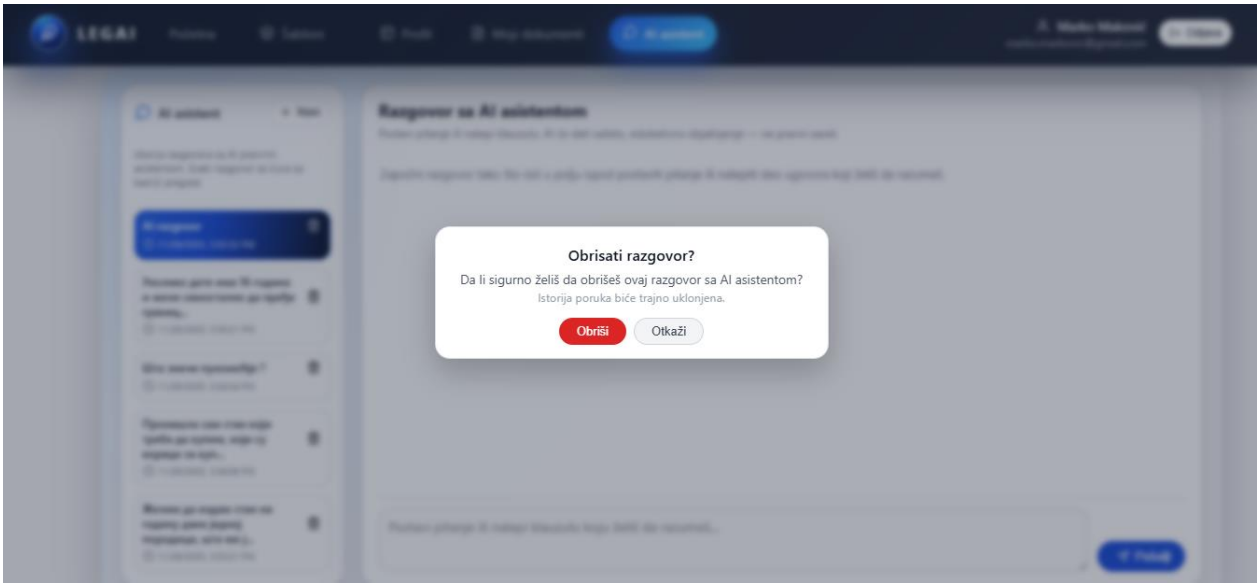
9.8 AI асистент

Кликом на страницу „*AI Asistent*“ у навигационом бару отвара се чет за комуникацију са вештачком интелигенцијом. Са леве стране налази се бочна трака са историјом свих претходних разговора, где се сваки појединачни чет аутоматски чува ради каснијег прегледа, враћања на претходна објашњења или настављања започетих тема. Кликом на дугме „*Novi*“ отвара се нови чет. Корисник може да кликне на било који претходни разговор како би га поново отворио, прегледао или наставио, као и да обрише разговор који му више није потребан. У централном делу странице налази се активно поље за тренутни разговор, у ком се приказују питања и одговори *AI* асистента у реалном времену. На дну странице налази се поље за унос поруке, а слање поруке врши се притиском на дугме „*Pošalji* “. На слици 9.8.1. је приказан *chat* са *AI* асистентом.



Слика 9.8.1. *Приказ chat-а са AI асистентом.*

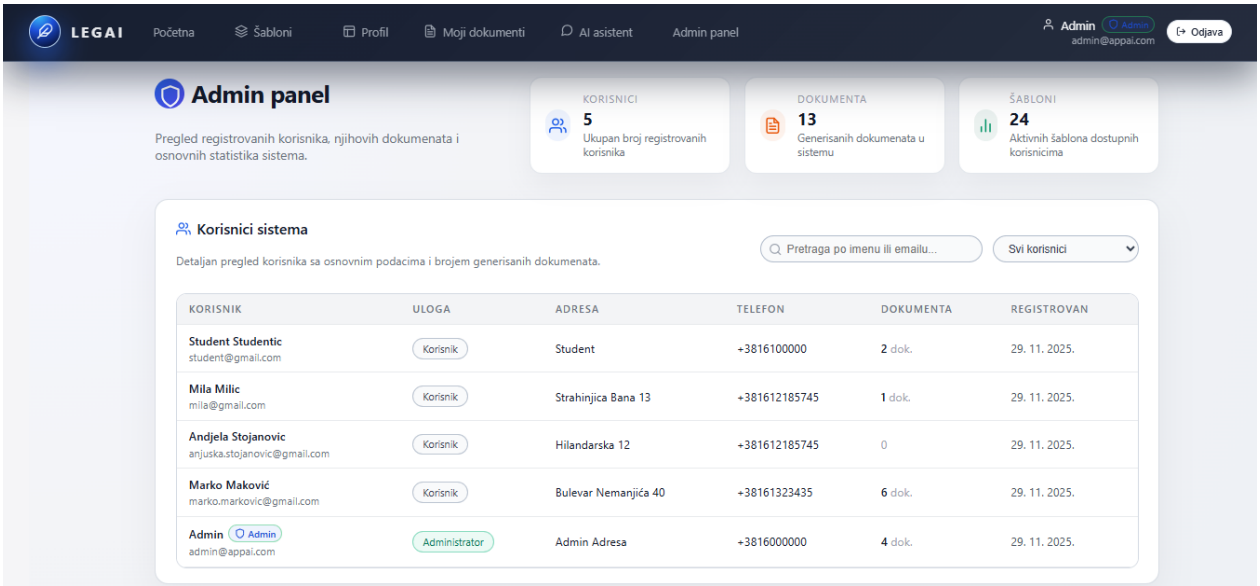
Могуће је трајно и неповратно обрисати чет са *AI* асистентом. Кликом на кантицу поред чета у историји свих разговора, отвара се прозор за брисање чета. Кликом на дугме „*Obriši*“ на прозору чет ће бити трајно обрисан из система. На слици 9.8.2. је приказ прозора за брисање чета са *AI* асистентом.



Слика 9.8.2. *Приказ прозора за брисање чета са AI асистентом.*

9.9. Администраторски панел

Администратор у свом навигационом бару има додатну страницу, а то је „*Admin panel*“. Администраторски панел служи за праћење основних статистичких података система. Приказане су кључне статистике: укупан број регистрованих корисника, укупан број генерисаних докумената и број активних шаблона доступних у систему. Испод статистичких картица налази се секција „*Korisnici sistema*“, у којој се за сваког корисника приказују основни подаци — име, имејл адреса, улога у систему (корисник или администратор), адреса, контакт телефон, укупан број докумената које је корисник генерисао, као и датум регистрације. Панел садржи и претрагу по имену или имејлу, као и филтер за избор категорије корисника (корисник или администратор), што омогућава брзо проналажење специфичних налога. На слици 9.9.1. је приказан администраторски панел.



Слика 9.9.1. *Приказ администраторског панела.*

10. Закључак

Развој веб решења *LegAI* показао је да комбинација модерних софтверских технологија и вештачке интелигенције може суштински да унапреди процес израде правних докумената и значајно подигне ниво правне писмености код грађана. Многи корисници, посебно они без формалног правног образовања, неретко се сусрећу са сложеним правним појмовима, неразумљивим формалним структурама и недоумицама око самог процеса попуњавања докумената. *LegAI* се показао као решење које ове баријере успешно превазилази: систем на једноставан и интуитиван начин води корисника кроз све кораке, пружа објашњења правних одредби, обједињује потребне елементе документа и омогућава креирање исправних и структурално коректних докумената без потребе за дубљим правним предзнањем.

У техничком смислу, реализован је стабилан и заокружен систем који обухвата *ASP.NET Core backend, React* кориснички интерфејс, *SQL Server* релациону базу података, *OpenXML* за креирање *DOCX* докумената, као и *OpenAI API* за интелигентну подршку кориснику. Интеграција ових технологија показује колико добро осмишљена архитектура може да уједини различите технолошке концепте у функционалну целину. Током рада на систему потврђено је да је могуће изградити решење које истовремено задовољава високе техничке стандарде, али и потребу за практичношћу, доступношћу и једноставном употребом код шире популације. *LegAI* представља пример да софтвер може бити развијен тако да директно утиче на реалне проблеме корисника, посебно у областима у којима недостаје приступачна дигитална подршка.

Иако је *LegAI* у садашњем облику функционалан и спреман за коришћење, простор за његов даљи развој је значајан. Систем се може унапредити увођењем аутоматске конверзије *DOCX* докумената у *PDF* формат, чиме би се корисницима пружио већи избор излазних формата и повећала флексибилност у раду. Додатно, развој вишејезичне подршке, интеграција електронског потписивања, повезивање са званичним правним регистрима и унапређење *AI* модула који би могао да анализира и предлаже измене у документима представљају логичан правац даљег истраживања и надоградње система. *LegAI*, као резултат овог рада, не само да демонстрира техничку оправданост и функционалност оваквог приступа, већ и поставља чврсту основу за будућа решења у области правне технологије и дигиталне подршке грађанима.

11. Референце

[1] Microsoft, “SQL Server Documentation,” [Online]. Available: https://learn.microsoft.com/sr-latn-rs/sql/sql-server/what-is-sql-server?view=sql-server-ver15, датум приступа: 3.11.2025. године.

[2] Microsoft, “ASP.NET Core Documentation,” [Online]. Available: https://dotnet.microsoft.com/en-us/learn/aspnet/what-is-aspnet, датум приступа: 3.11.2025. године.

[3] React Team, “React Documentation,” [Online]. Available: https://react.dev/reference/react?utm_source=chatgpt.com, датум приступа: 3.11.2025. године.

[4] Microsoft, “OpenXML SDK Documentation,” [Online]. Available: https://learn.microsoft.com/office/open-xml/open-xml-sdk, датум приступа: 5.11.2025. године.

[5] OpenAI, “OpenAI API Documentation,” [Online]. Available: https://platform.openai.com/docs, датум приступа: 7.11.2025. године.

[6] SlideModel, “Three-Tier Architecture Model – PowerPoint Template,” SlideModel.com. [Online]. Available: https://slidemodel.com/templates/three-tier-architecture-model-powerpoint-template/, датум приступа: 9.11.2025. године.

[7] Brainspate, “Frontend vs Backend in eCommerce – Unlocking Web Development Secrets,” Brainspate.com. [Online]. Available: https://brainspate.com/blog/frontend-vs-backend-in-ecommerce-unlocking-web-development-secrets/, датум приступа: 9.11.2025. године.

[8] Microsoft, “What is ASP.NET?,” [Online]. Available: https://dotnet.microsoft.com/en-us/learn/aspnet/what-is-aspnet, датум приступа: 10.11.2025. године.

[9] Microsoft, “ASP.NET Core,” [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/introduction-to-aspnet-core?view=aspnetcore-9.0, датум приступа: 12.11.2025. године.

[10] Microsoft, “Overview of ASP.NET Core authentication,” [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/?view=aspnetcore-9.0, датум приступа: 13.11.2025. године.

[11] Microsoft, “Introduction to authorization in ASP.NET Core,” [Online]. Available:

https://learn.microsoft.com/en-us/aspnet/core/security/authorization/introduction?view=aspnetcore-9.0., датум приступа: 14.11.2025. године.

[12] Microsoft, “Introduction to Identity on ASP.NET Core,” [Online]. Available: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity?view=aspnetcore-9.0&tabs=visual-studio., датум приступа: 16.11.2025. године.

[13] “Learn ASP.NET Web API,” [Online]. Available: https://www.tutorialsteacher.com/webapi, датум приступа: 17.11.2025. године.

[14] iDempiere, “Understanding REST API and Its Importance in iDempiere Development,” iDempiere.org. [Online]. Available: https://idempiere.org/blog/2025/03/27/understanding-rest-api-and-its-importance-in-idempiere-development/, датум приступа: 17.11.2025. године.

[15] Microsoft, “What is MSSQL Server?,” [Online]. Available: https://hasura.io/learn/database/microsoft-sql-server/what-is-mssql/, датум приступа: 18.11.2025. године.

[16] OpenAI, “API Pricing,” [Online]. Available: https://openai.com/api/pricing/. датум приступа: 7.11.2025. године.

[17] OpenAI, “OpenAI Models – Overview,” [Online]. Available: https://platform.openai.com/docs/models, датум приступа: 7.11.2025. године.

ИЗЈАВА О САМОСТАЛНОСТИ И ОРИГИНАЛНОСТИ ИЗРАДЕ РАДА

Изјављујем да је *завршни / дипломски/ мастер* рад, под насловом:

Веб решење за аутоматско генерисање и управљање правним документима уз асистенцију вештачке интелигенције

- резултат сопственог истраживачког рада;

- да нисам повредио/ла ауторска права, нити злоупотребио/ла интелектуалну својину других лица.

У Нишу, 01. 12. 2025.

Потпис аутора/ке

Анђела Стојановић

(име, презиме)

18 406

(број индекса)